

Lista de palabras clave

ABS

Sintaxis: ABS (<expresión numérica>)

Ejemplo:

```
PRINT ABS(-67.98)
67.98
```

Función. Calcula el valor absoluto de la expresión dada; es decir, convierte los números negativos en positivos.

Palabras clave asociadas: SGN.

AFTER

Sintaxis: AFTER <retardo del temporizador >[, <número de temporizador>] GOSUB [número de línea]

Ejemplo:

```
10 AFTER 250 GOSUB 60:CLS
20 PRINT "Adivine una letra en 5 segundos"
30 a$=INKEY$:IF indicador=1 THEN END
40 IF a$<>CHR$(INT(RND*26+97)) THEN 30
50 PRINT a$;" es correcto. Usted gana."
55 SOUND 1,478:SOUND 1,358:END
60 PRINT "demasiado tarde. Yo gano."
70 SOUND 1,2000:indicador=1:RETURN
run
```

Orden. Invoca una subrutina de BASIC cuando ha transcurrido el tiempo especificado. El parámetro <retardo del temporizador> especifica el tiempo en unidades de 0.02 s (cincuentavos de segundo).

El <número de temporizador> (margen de 0 a 3) especifica cuál de los cuatro temporizadores disponibles debe medir el tiempo. El temporizador número 3 tiene la máxima prioridad; el 0 (que es el implícito), la mínima.

Cada temporizador puede tener una subrutina asociada.

Para más amplia información sobre las interrupciones, consulte la parte 3 del capítulo titulado 'Cuando usted guste ...'.

Palabras clave asociadas: EVERY, REMAIN, RETURN.

AND

Sintaxis: (argumento) AND (argumento)

Ejemplo:

```

IF "juan"<"pepe" AND "perro">"gato" THEN PRINT "verdadero" ELSE PRINT "falso"
verdadero
IF "pepe"<"juan" AND "gato">"perro" THEN PRINT "verdadero" ELSE PRINT "falso"
falso
IF "juan"<"pepe" AND "gato">"perro" THEN PRINT "verdadero" ELSE PRINT "falso"
Falso
...
PRINT 1 AND 1
1
PRINT 0 AND 0
0
PRINT 1 AND 0
0

```

Operador. Realiza la operación lógica 'y' bit a bit entre enteros. El resultado es 0 a menos que ambos bits sean 1.

Para más amplia información sobre operaciones lógicas, consulte la parte 2 del capítulo titulado 'Cuando usted guste...'

Palabras clave asociadas: OR, NOT, XOR.

ASC

Sintaxis: ASC (<expresión literal>)

Ejemplo:

```

PRINT ASC("x")
120

```

Función. Calcula el código ASCII del primer carácter de la <expresión literal>.

Palabras clave asociadas: CHR\$.

ATN

Sintaxis: ATN (<expresión numérica>)

Ejemplo:

```

PRINT ATN(1)
0.785398163

```

Función. Calcula el arco cuya tangente es la <expresión numérica>.

Obsérvese que se puede utilizar DEG y RAD para hacer que el resultado se dé en grados o en radianes, respectivamente.

Palabras clave asociadas: COS, DEG, RAD, SIN, TAN.

AUTO

Sintaxis: AUTO [<número de línea>] [, <incremento>]

Ejemplo:

```

AUTO 100,50

```

Orden. Genera números de línea AUTOMáticamente. El parámetro opcional <número de línea> especifica en qué número debe empezar la numeración. Si se lo omite, el ordenador genera números de línea a partir del 10.

El parámetro opcional <incremento> especifica el salto que debe haber entre cada línea y la siguiente. Si se lo omite, el ordenador genera números de línea espaciados de 10 en 10.

Si se genera un número de línea que ya ha sido utilizado, el ordenador la muestra en la pantalla y entonces se la puede editar, si es necesario. La línea editada sustituye a la que está en la memoria en el momento de pulsar [RETURN].

Para interrumpir el proceso de numeración automática, pulse [ESC].

Palabras clave asociadas: ninguna.

BIN\$

Sintaxis: BIN\$ (<expresión entera> [, <expresión entera>])

Ejemplo:

```
PRINT BIN$(64,8)
01000000
```

Función. Produce una cadena de dígitos binarios que representan el valor de la primera <expresión entera>, utilizando el número de dígitos binarios especificados por la segunda <expresión entera> (margen de 0 a 16). Si el número de dígitos especificado es mayor que el necesario, la expresión resultante se completa con ceros por la izquierda; si el número de dígitos especificado es demasiado pequeño, la expresión resultante no se trunca, sino que contiene todos los dígitos que sean necesarios.

La <expresión entera> que se va a convertir en binario debe tener un valor comprendido entre -32768 y + 65535.

Palabras clave asociadas: DEC\$, HEX\$, STR\$.

BORDER

Sintaxis: BORDER <color>[, <color>]

Ejemplo:

```
10 REM 729 combinaciones para el borde!
20 SPEED INK 5,5
30 FOR a=0 TO 26
40 FOR b=0 TO 26
50 BORDER a,b:CLS:LOCATE 14,13
60 PRINT "border";a;" ";b
70 FOR t=1 TO 500
80 NEXT t,b,a
run
```

Orden. Cambia el color del borde de la pantalla. Si se especifican dos colores, el del borde alterna entre ambos, a una velocidad que se puede regular con la orden SPEED INK. El margen de colores es de 0 a 26.

Palabras clave asociadas: SPEED INK.

BREAK

Véase ON BREAK CONT, ON BREAK GOSUB, ON BREAK STOP.

CALL

Sintaxis: CALL <expresión de dirección de memoria>[, <lista de:<parámetro>]

Ejemplo:

```
CALL 0
```

Orden. Permite invocar desde BASIC una rutina escrita en código de máquina. El ejemplo anterior provoca la reinicialización de la máquina.

Esta orden no debe ser utilizada si no se sabe cuáles son sus efectos. *Palabras clave asociadas:* UNT.

CAT

Sintaxis: CAT

Ejemplo:

```
CAT
```

Orden. Da un catálogo del contenido del disco. Presenta en la pantalla, en orden alfanumérico, los nombres completos de todos los ficheros, junto con las respectivas longitudes (en unidades de Kbytes, redondeadas por exceso). También se muestra el espacio libre que queda en el disco, así como las identificaciones de la unidad y del usuario.

Esta orden no afecta al programa actualmente residente en la memoria.

Palabras clave asociadas: LOAD, RUN, SAVE.

Nota del revisor (2008): En los CPC464, o si se usa un CPC664 o un CPC6128 leyendo desde una unidad de cassette, esta orden hace que BASIC se ponga a examinar la cinta y dé una lista de los ficheros que va encontrando. Para cada fichero muestra cada uno de los bloques integrantes, con el siguiente formato:

```
NOMBREDEFICHERO Númerodebloque Indicador Ok
```

Se verá más detalle sobre el manejo de cintas en el capítulo 4, parte 2.

CHAIN

Sintaxis: CHAIN <nombre de fichero>[,<expresión de número de línea>]

Ejemplo:

```
CHAIN "programa.bas",350
```

Orden. Lee un programa grabado en disco y lo carga en la memoria reemplazando al actual. Inicia la ejecución del nuevo programa, bien desde el principio, bien desde una línea especificada por el parámetro opcional <expresión de número de línea>.

Los ficheros protegidos (grabados con SAVE ... ,p) se pueden cargar y ejecutar con esta orden.

Palabras clave asociadas: CHAIN MERGE, LOAD, MERGE.

CHAIN MERGE

Sintaxis: CHAIN MERGE <nombre de fichero>[,<expresión de número de línea>] [, DELETE <margen de números de línea>]

Ejemplo:

```
CHAIN MERGE "nuevopr.p.bas",750,DELETE 400-680
```

Orden. Lee un programa grabado en el disco, lo carga en la memoria mezclándolo con el actual y empieza a ejecutar el programa resultante, bien desde el principio, bien desde una línea especificada por el parámetro opcional <expresión de número de línea>. Si se desea borrar parte del programa original antes de iniciar la carga del nuevo, se puede utilizar la opción DELETE <margen de números de línea>.

Obsérvese que las líneas del programa que se carga sustituyen a las que tuvieran el mismo número en el programa original.

Los ficheros protegidos (grabados con SAVE ... ,p) no se pueden cargar con esta orden.

Palabras clave asociadas: CHAIN, DELETE, LOAD, MERGE.

CHR\$

Sintaxis: CHR\$ (<expresión entera>)

Ejemplo:

```
10 FOR x=32 TO 255
20 PRINT x;CHR$(x),
30 NEXT
run
```

Función. Convierte la <expresión entera>, que tiene que estar en el margen de 0 a 255, en su carácter equivalente. El conjunto de caracteres del AMSTRAD se muestra en la parte 3 del capítulo titulado 'Para su referencia'.

Obsérvese que los números 0 a 31 generan caracteres de control; por esa razón los hemos excluido del ejemplo anterior.

Palabras clave asociadas: ASC.

CINT

Sintaxis: CINT (<expresión numérica>)

Ejemplo:

```
10 n=1.9999
20 PRINT CINT(n)
run
2
```

Función. Aproxima el valor de la <expresión numérica> a un número entero comprendido en el margen de -32768 a +32767.

Palabras clave asociadas: CREAL, FIX, INT, ROUND, UNT.

CLEAR

Sintaxis: CLEAR

Ejemplo:

```
CLEAR
```

Orden. Borra todas las variables (igual a cero las numéricas y a la cadena vacía las literales). Además, se abandonan los ficheros que estén abiertos, se borran todas las listas y funciones definidas por el usuario y BASIC retorna al modo de radianes para el cálculo de funciones trigonométricas.

Palabras clave asociadas: ninguna.

CLEAR INPUT

Sintaxis: CLEAR INPUT

Ejemplo:

```
10 CLS
20 PRINT "Teclee letras ahora!"
30 FOR t=1 TO 3000
40 NEXT
50 CLEAR INPUT
run
```

Orden. Desecha todos los caracteres tecleados antes y que todavía estén en el tampón del teclado; es decir, borra el tampón del teclado.

Para observar el efecto de esta orden, ejecute el programa ejemplo y escriba letras cuando el ordenador se lo pida. Borre luego la línea 50 y vuelva a ejecutar el programa.

Palabras clave asociadas: INKEY, INKEY\$, JOY.

CLG

Sintaxis: CLG [<tinta>]

Ejemplo:

```
LOCATE 1,220
CLG 3
```

Orden. Borra la pantalla gráfica, dejándola del color del papel de gráficos. Si se especifica la <tinta>, el papel de gráficos toma ese color.

Palabras clave asociadas: CLS, GRAPHICS PAPER, INK, ORIGIN.

CLOSEIN

Sintaxis: CLOSEIN

Ejemplo:

```
CLOSEIN
```

Orden. Cierra el fichero de disco que esté abierto en dirección de entrada (si lo hay). (Véase OPENIN.)

Palabras clave asociadas: CLOSEOUT, EOF, OPENIN.

CLOSEOUT

Sintaxis: CLOSEOUT

Ejemplo:

```
CLOSEOUT
```

Orden. Cierra el fichero de disco que esté abierto en dirección de salida (si lo hay). (Véase OPENOUT.)

Palabras clave asociadas: CLOSEIN, OPENOUT.

CLS

Sintaxis: CLS [# <expresión de número de canal>]

Ejemplo:

```
10 PAPER #2,3
20 CLS #2
run
```

Orden. Borra la ventana (canal de pantalla) especificada, tiñéndola del color del papel. Si no se especifica ninguna <expresión de número de canal>, el ordenador borra la pantalla #0.

Palabras clave asociadas: CLG, INK, PAPER, WINDOW.

CONT

Sintaxis: CONT

Ejemplo:

```
CONT
```

Orden. Reanuda la ejecución del programa cuando éste ha sido interrumpido, bien porque se ha pulsado la tecla [ESC] dos veces, bien porque el programa ha encontrado la instrucción STOP. CONT solamente puede continuar el programa en el supuesto de que no se lo haya modificado durante la interrupción y de que no se trate de un programa protegido.

Palabras clave asociadas: STOP.

COPYCHR\$

Sintaxis: COPYCHR\$(# <expresión de número de canal>)

Ejemplo:

```
10 CLS
20 PRINT "Extremo superior izquierdo"
30 LOCATE 1,1
40 a$=COPYCHR$(#0)
50 LOCATE 1,20
60 PRINT a$
run
```

Función. Lee el carácter que hay en la posición actual del cursor; la ventana tiene que ser especificada. El programa ejemplo lee el carácter que está en la posición 1,1 y lo escribe en la

posición 1,20.

Si el carácter no es reconocible, esta función genera la cadena vacía.

Palabras clave asociadas: LOCATE.

Nota del revisor (2008): La función COPYCHR\$ está disponible en los Amstrad CPC6128 y en los CPC664, pero no en el modelo inicial CPC464.

COS

Sintaxis: COS (<expresión numérica>)

Ejemplo:

```
DEG
PRINT COS(45)
0.707106781
```

Función. Calcula el coseno de la <expresión numérica>.

Obsérvese que se puede utilizar DEG y RAD para hacer que el ordenador entienda los arcos como dados en grados o en radianes, respectivamente.

Palabras clave asociadas: ATN, DEG, RAD, SIN.

CREAL

Sintaxis: CREAL (<expresión numérica>)

Ejemplo:

```
10 a=PI
20 PRINT CINT(a)
30 PRINT CREAL(a)
run
3
3.14159265
```

Función. Convierte el valor de la <expresión numérica>, cualquiera que sea su tipo, en un número real.

Palabras clave asociadas: CINT.

CURSOR

Sintaxis: CURSOR (<conmutador del sistema>)[, <conmutador del usuario>]

Ejemplo:

```
10 CURSOR 1
20 PRINT "pregunta?";
30 a$=INKEY$;IF a$="" THEN 30
40 PRINT a$
50 CURSOR 0
run
```

Orden. El cursor puede ser activado o inhibido por dos conmutadores: el del sistema y el del usuario. Esta orden controla los dos conmutadores. Los parámetros pueden ser 0 (inhibido) o 1 (activado). En la instrucción INKEYS el cursor está normalmente oculto, pero en el programa

ejemplo ha sido activado por el valor 1 del parámetro <conmutador del sistema> en la línea 10.

Para que el cursor sea visible tienen que estar a 1 los dos conmutadores. El ordenador activa automáticamente el conmutador del sistema para la instrucción INPUT, pero lo desactiva para INKEY\$.

Es conveniente ocultar el cursor antes de escribir texto en la pantalla.

Se puede omitir cualquiera de los parámetros, pero no los dos. Cuando se omite uno de los parámetros, la situación del correspondiente conmutador no se modifica.

Palabras clave asociadas: LOCATE.

Nota del revisor (2008): La orden CURSOR está disponible en los Amstrad CPC6128 y en los CPC664, pero no en el modelo inicial CPC464.

DATA

Sintaxis: DATA <lista de:<constante>>

Ejemplo:

```
10 FOR x=1 TO 4
20 READ nombre$,apellido$
30 PRINT "Don ";nombre$;" ";apellido$
40 NEXT
50 DATA Manuel,Gonzalez,Daniel,Garcia
60 DATA Felipe,Revilla,Ernesto,Diaz
run
```

Orden. Identifica las líneas en las que están contenidos los datos que pueden ser leídos con la instrucción READ. Cada vez que se lee un dato, el 'puntero' pasa a señalar el siguiente elemento de la lista. Se puede utilizar la instrucción RESTORE para hacer que el puntero señale una línea de datos determinada.

En la parte 2 del capítulo titulado 'Cuando usted guste ...' daremos más amplia información sobre los datos de este tipo.

Palabras clave asociadas: READ, RESTORE.

DEC\$

Sintaxis: DEC\$ (<expresión numérica> , <plantilla de formato>)

Ejemplo:

```
PRINT DEC$(10E7,"PtPt#####,.##")
Pt10,000,000.00
```

Función. Produce una representación en forma de cadena decimal de la <expresión numérica> utilizando la <plantilla de formato> especificada.

La plantilla de formato sólo puede contener los caracteres siguientes:

+ - Pt \$ # , . ^

La utilización de estos 'especificadores de formato' se describe en PRINT USING.

Palabras clave asociadas: BIN\$, HEX\$, PRINT USING, STR\$.

DEF FN

Sintaxis: DEF FN <nombre de la función> [(<parámetros de forma>)] = <expresión>

Ejemplo:

```
10 t=TIME/300
20 DEF FNreloj=INT(TIME/300-t)
30 EVERY 100 GOSUB 50
40 GOTO 40
50 PRINT "el programa se ejecuto hace";
60 PRINT FNreloj;"segundos"
70 RETURN
run
```

Orden. BASIC permite que el programa defina y utilice funciones. DEF FN realiza la primera parte de ese mecanismo: crea una función específica del programa; una vez definida, la función se maneja de la misma forma que las funciones intrínsecas de BASIC.

(Nótese en el ejemplo anterior que el valor de la función FNreloj sigue siendo actualizada ininterrumpidamente, aunque se imponga una pausa al programa pulsando [ESC] dos veces para luego reanudarlo con CONT.)

Palabras clave asociadas: ninguna.

DEFINT

Sintaxis: DEFINT <lista de: <margen de letras> >

Ejemplo:

```
10 DEFINT n
20 numero=123.456
30 PRINT numero
run
123
```

Orden. Cada vez que el programa encuentra una variable que no vaya seguida de un indicador explícito de tipo (! % \$), el ordenador considera que es del tipo implícito. Esta orden hace que el tipo implícito sea 'variable entera'. La orden afecta a todas las variables cuyos nombres empiecen por la letra o letras especificadas. Se puede especificar una lista:

```
DEFINT a,b,c
```

o bien un margen de iniciales:

```
DEFINT a-z
```

Palabras clave asociadas: DEFREAL, DEFSTR.

DEFREAL

Sintaxis: DEFREAL <lista de: <margen de letras> >

Ejemplo:

```
DEFREAL x,a-f
```

Orden. Cada vez que el programa encuentra una variable que no vaya seguida de un indicador

explícito de tipo (! % \$), el ordenador considera que es del tipo implícito. Esta orden hace que el tipo implícito sea 'variable real'. La orden afecta a todas las variables cuyos nombres empiecen por la letra o letras especificadas. Se puede especificar una lista:

```
DEFREAL a, b, c
```

o bien un margen de iniciales:

```
DEFREAL a-z
```

Palabras clave asociadas: DEFINT, DEFSTR.

DEFSTR

Sintaxis: DEFSTR <lista de:<margen de letras> >

Ejemplo:

```
10 DEFSTR n
20 nombre="Amstrad"
30 PRINT nombre
run
Amstrad
```

Orden. Cada vez que el programa encuentra una variable que no vaya seguida de un indicador explícito de tipo (! % \$), el ordenador considera que es del tipo implícito. Esta orden hace que el tipo implícito sea 'variable literal'. La orden afecta a todas las variables cuyos nombres empiecen por la letra o letras especificadas. Se puede especificar una lista:

```
DEFSTR a, b, c
```

bien un margen de iniciales:

```
DEFSTR a-z
```

Palabras clave asociadas: DEFINT, DEFREAL.

DEG

Sintaxis: DEG

Ejemplo:

```
DEG
```

Orden. Hace que BASIC considere que los valores de los arcos que intervienen en los cálculos trigonométricos están expresados en grados, no en radianes. La situación implícita para las funciones SIN, COS, TAN y ATN es la inversa: radianes. DEG cambia de radianes a grados; las órdenes RAD, NEW, CLEAR, LOAD, RUN, etc. cambian de grados a radianes.

Palabras clave asociadas: ATN, COS, RAD, SIN, TAN.

DELETE

Sintaxis: DELETE <margen de números de línea>

Ejemplo:

```
DELETE 100-200
```

Orden. Borra la instrucción o grupo de instrucciones especificado por <margen de números de línea>.

Si se omite el primer número de <margen de números de línea>, se entiende 'desde el principio del programa hasta ...'; si se omite el segundo número, se entiende '... hasta el final del programa'. Así,

```
DELETE -200
```

borra las primeras líneas del programa, hasta la 200 inclusive. En cambio,

```
DELETE 50-
```

borra desde la línea 50 en adelante. Finalmente,

```
DELETE
```

borra el programa completo.

Palabras clave asociadas: CHAIN, MERGE, RENUM.

DERR

Sintaxis: DERR

Ejemplo:

```
LOAD "xyz.abc"  
XYZ .ABC not found Ready  
PRINT DERR  
146
```

Función. Toma como valor el último código de error detectado por el sistema de archivo en disco. Se puede utilizar el valor de DERR para determinar qué error se ha producido. Véase en el capítulo 'Para su referencia ...' la lista de los mensajes de error.

Palabras clave asociadas: ERL, ERR, ERROR, ON ERROR GOTO, RESUME.

DI

Sintaxis: DI

Ejemplo:

```
10 CLS:TAG:EVERY 10 GOSUB 90  
20 X1=RND*320: X2=RND*320  
30 Y=200+RND*200:C$=CHR$(RND*255)  
40 FOR X=320-X1 TO 320+X2 STEP 4  
50 DI  
60 MOVE 320,0,1:MOVE X-2,Y:MOVE X,Y  
70 PRINT " ";C$;:FRAME  
80 EI:NEXT:GOTO 20  
90 MOVE 320,0:DRAW X+8,Y-16,0:RETURN  
run
```

Orden. Inhibe interrupciones (a excepción de [ESC]) hasta que se las habilite explícitamente con EI o implícitamente con el RETURN en el que termina la subrutina invocada como consecuencia de la interrupción.

Nótese que al entrar en una rutina de interrupción se inhiben automáticamente todas las interrupciones de prioridad igual o inferior.

Esta orden se utiliza para asegurar que el programa funcione sin interrupción; por ejemplo, cuando dos rutinas están compitiendo por el uso de recursos comunes. En el ejemplo anterior, el programa principal y la subrutina de interrupción están compitiendo por el uso de la pantalla gráfica.

En la parte 3 del capítulo 'Cuando usted guste...' daremos más amplia información a este respecto.

Palabras clave asociadas: AFTER, EI, EVERY, REMAIN.

DIM

Sintaxis: DIM <lista de:<variable indexada>>

Ejemplo:

```
10 CLS
20 DIM amigo$(5),telefono$(5)
30 FOR n=1 TO 5
40 PRINT "Amigo numero: ";n
50 INPUT "Nombre ";amigo$(n)
60 INPUT "Telefono ";telefono$(n)
70 PRINT
80 NEXT
90 FOR n=1 TO 5
100 PRINT n;amigo$(n),telefono$(n)
110 NEXT
run
```

Orden. DIMensiona listas. DIM hace una reserva de espacio para los elementos de la lista y especifica cuál va a ser el máximo valor del índice. BASIC requiere que se le informe del tamaño que debe reservar para las listas; sin embargo, permite el manejo de listas no dimensionadas (es decir, no declaradas con una orden DIM) a condición de que el índice no sobrepase el valor 10.

Una lista es una 'variable indexada', lo que quiere decir que se utiliza un solo nombre de variable para representar varios elementos; cada elemento está identificado por su 'índice'. Las listas pueden ser controladas fácilmente mediante bucles de tipo FOR ... NEXT, identificando el índice con la variable del contador del bucle.

Obsérvese que el valor mínimo del índice (esto es, el primer elemento utilizable) es el cero.

Un vector es una lista monodimensional. También se pueden utilizar listas multidimensionales (matrices). Por ejemplo, una lista tridimensional se dimensionaría con

```
DIM posicion$(20,20,20)
```

y un elemento concreto sería

```
posicion$ (4,5,6)
```

Palabras clave asociadas: ERASE.

DRAW

Sintaxis: DRAW <coordenada x>, <coordenada y>[, [<tinta>][, <modo de tinta>]]

Ejemplo:

```
10 MODE 0:BORDER 0:PAPER 0:INK 0,0
20 x=RND*640:y=RND*400:z=RND*15
30 DRAW x,y,z
40 GOTO 20
run
```

Orden. Dibuja en la pantalla gráfica una recta que va desde la posición actual del cursor gráfico hasta el punto de coordenadas x, y especificadas. Se puede especificar el color de <tinta> con el que se desea dibujar la recta (margen: 0 a 15).

El <modo de tinta> es opcional y determina el modo de interacción entre la tinta con que se dibuja y la que ya está en la pantalla gráfica. Los cuatro modos son los siguientes:

```
0 Normal
1 XOR (OR exclusivo)
2 AND
3 OR
```

Palabras clave asociadas: DRAWR, GRAPHICS PEN, MASK.

DRAWR

Sintaxis: DRAWR <incremento de x>, <incremento de y>[, [<tinta>][, <modo de tinta>]]

Ejemplo:

```
10 CLS:PRINT "escalera"
20 MOVE 0,350:FOR n=1 TO 8
30 DRAWR 50,0
40 DRAWR 0,-50
50 NEXT:MOVE 348,0:FILL 3
60 GOTO 60
run
```

Orden. Dibuja en la pantalla gráfica una recta que va desde la posición actual del cursor gráfico hasta el punto especificado en coordenadas relativas por <incremento de x> e <incremento de y>. Se puede especificar el color de <tinta> con el que se desea dibujar (margen: 0 a 15).

El <modo de tinta> es opcional y determina el modo de interacción entre la tinta con que se dibuja y la que ya está en la pantalla gráfica. Los cuatro modos son los siguientes:

```
0 Normal
1 XOR (OR exclusivo)
2 AND
3 OR
```

Palabras clave asociadas: DRAW, GRAPHICS PEN, MASK.

EDIT

Sintaxis: EDIT <número de línea>

Ejemplo:

```
EDIT 20
```

Orden. Escribe en la pantalla la línea especificada por <número de línea> y la deja dispuesta para ser editada, con el cursor sobre ella.

Palabras clave asociadas: AUTO, LIST.

EI

Sintaxis: EI

Ejemplo:

EI

Orden. Habilita interrupciones que hayan sido inhibidas previamente con DI.

Si se inhiben las interrupciones en una subrutina de interrupción, BASIC las habilita cuando encuentra el RETURN del final de la subrutina.

En la parte 3 del capítulo titulado 'Cuando usted guste...' daremos más amplia información acerca de las interrupciones.

Palabras clave asociadas: AFTER, DI, EVERY, REMAIN.

ELSE

(Véase IF.)

END

Sintaxis: END

Ejemplo:

END

Orden. Termina la ejecución del programa y devuelve el ordenador al modo directo. En un mismo programa se pueden incluir varias órdenes END; se supone un END implícito después de la última línea del programa.

Palabras clave asociadas: STOP.

ENT

Sintaxis: ENT <número de envolvente> [, <sección de envolvente>]

[, <sección de envolvente>][, <sección de envolvente>]

[, <sección de envolvente>][, <sección de envolvente>]

Ejemplo:

```
10 ENT 1,10,-50,10,10,50,10
20 SOUND 1,500,200,10,,1
run
```

Orden. Define la características de la envolvente de tono especificada por <número de envolvente> (margen: de 1 a 15) e invocada como <envolvente de tono> en la orden **SOUND**. Si el <número de envolvente> es negativo <margen de -1 a -15>, la envolvente se repite hasta el final de la <duración> especificada en la orden SOUND.

Cada <sección de envolvente> puede constar de 2 o 3 parámetros. Si se utilizan 3 parámetros, tienen el significado siguiente:

<número de escalones>, <altura de cada escalón>, <tiempo de pausa>

Parámetro 1: <número de escalones>

Este parámetro especifica en cuántas etapas queremos que la nota evolucione a lo largo de la duración de la sección de la envolvente. Por ejemplo, si una sección de la nota ha de durar 10 segundos, se la puede dividir en 10 etapas de 1 segundo cada una; en tal caso, el parámetro <número de escalones> tendrá que ser 10.

El margen de este parámetro es de 0 a 239.

Parámetro 2: <altura de cada escalón>

En cada etapa, el tono puede variar con respecto al de la anterior en un número de unidades comprendido entre -128 y +127. Las variaciones negativas representan aumentos del tono (tono más agudo); las variaciones positivas reducen el tono (tono más grave). El valor mínimo del período de tono es 0. En el capítulo 'Para su referencia ...' se da la lista completa de los periodos de tono.

Parámetro 3: <tiempo de pausa>

Este parámetro especifica la duración de cada escalón (intervalo entre cada dos saltos) en unidades de centésimas de segundo. El margen de valores es de 0 a 255; esto significa que la duración máxima de cada escalón es de 2.56 segundos (el 0 se considera como 256).

Si se utilizan 2 parámetros, su significado es el siguiente:

<periodo de tono>, <tiempo de pausa>

Parámetro 1: <periodo de tono>

Este parámetro da un nuevo valor absoluto del periodo de tono. (Véase el parámetro 2 de la orden SOUND.)

Parámetro 2: <tiempo de pausa>

Este parámetro especifica la duración de escalón en unidades de centésimas de segundo. El margen de valores es de 0 a 255, es decir, el valor máximo es 2.56 segundos, ya que el 0 se considera como 256.

General

La suma de todos los <tiempos de pausa> no debe ser mayor que el parámetro <duración> de la orden SOUND; de lo contrario, el sonido habrá concluido antes de que se hayan completado todas las etapas de la envolvente. (En tal caso, el ordenador ignora el resto del contenido de la envolvente.)

Análogamente, si la duración especificada en SOUND es mayor que la impuesta por la suma de todos los <tiempos de pausa>, la nota continuará sonando aunque se hayan terminado los escalones de la envolvente de tono, y lo hará con el tono correspondiente al de la última etapa de la envolvente.

En una orden ENT se puede definir un máximo de 5 <secciones de envolvente> (cada una de las cuales puede constar de 2 o 3 parámetros).

La primera etapa de una envolvente de tono se ejecuta inmediatamente.

Cada vez que se define una envolvente de tono, se anula la definición anterior.

Si se define un <número de envolvente> sin especificar parámetros para ninguna de sus secciones, se anula la definición anterior.

En la parte 2 del capítulo 'Cuando usted guste...' daremos más información sobre las envolventes

de tono.

Palabras clave asociadas: ENV, SOUND.

ENV

Sintaxis: ENV <número de envolvente>[, <sección de envolvente>)]

[, <sección de envolvente>)][, <sección de envolvente>]

[, <sección de envolvente>)][, <sección de envolvente>]

Ejemplo:

```
10 ENV 1,15,-1,10,15,1,10
20 SOUND 1,200,300,15,1
run
```

Orden. Define las características de la envolvente de volumen especificada por <número de envolvente> (margen: de 1 a 15) e invocada como <envolvente de volumen> en la orden SOUND.

Cada <sección de envolvente> puede constar de 2 o 3 parámetros.

Si se utilizan 3 parámetros, tienen el significado siguiente:

<número de escalones>, <altura de cada escalón>, <tiempo de pausa>

Parámetro 1: <número de escalones>

Este parámetro especifica en cuántas etapas queremos que la nota evolucione antes de completar la sección de la envolvente. Por ejemplo, si una sección de la nota ha de durar 10 segundos, se la puede dividir en 10 etapas de 1 segundo cada una; en tal caso, el parámetro <número de escalones> será 10.

El margen de este parámetro es de 0 a 127.

Parámetro 2: <altura de cada escalón>

En cada etapa, el volumen puede variar con respecto al nivel anterior en un número de unidades comprendido entre 0 y 15. Estos 16 niveles de sonido son los mismos que se definen en la instrucción SOUND. Sin embargo, el margen del parámetro <altura de cada escalón> es de — 128 a + 127; el nivel del volumen vuelve 0 cada vez que sobrepasa el 15.

Parámetro 3: <tiempo de pausa>

Este parámetro especifica la duración de cada escalón en unidades de centésimas de segundo. El margen de valores es de 0 a 255; esto significa que la duración máxima de cada escalón es de 2.56 segundos (el 0 se considera como 256).

Si se utilizan 2 parámetros, su significado es el siguiente:

<envolvente de hardware>, <periodo de envolvente>

Parámetro 1: <envolvente de hardware>

Este parámetro especifica el valor que se debe enviar al registro de forma de envolventes del generador de sonido.

Parámetro 2: <periodo de envolvente>

Este parámetro especifica el valor que se debe enviar a los registros de periodo de envolventes del generador de sonido.

Para utilizar las envolventes de hardware es necesario que el usuario conozca el funcionamiento interno del generador de sonido. De no ser así, le recomendamos que utilice las envolventes definidas por programa, incorporando un <tiempo de pausa> adecuado.

General

La suma de todos los <tiempos de pausa> no debe ser mayor que el parámetro <duración> de la orden SOUND; de lo contrario, el sonido habrá concluido antes de que se hayan completado todas las etapas de la envolvente. (En tal caso, el ordenador ignora el resto del contenido de la envolvente.)

Análogamente, si la duración especificada en SOUND es mayor que la impuesta por la suma de todos los <tiempos de pausa>, la nota continuará sonando aunque se hayan terminado los escalones de la envolvente de tono, y lo hará con nivel de volumen igual al de la última etapa de la envolvente.

En una orden ENV se puede definir un máximo de 5 <secciones de envolvente> (cada una de las cuales puede constar de 2 o 3 parámetros).

La primera etapa de una envolvente de volumen se ejecuta inmediatamente.

Cada vez que se define una envolvente de volumen, se anula la definición anterior.

Si se define un <número de envolvente> sin especificar parámetros para ninguna de sus secciones, se anula la definición anterior.

En la parte 2 del capítulo 'Cuando usted guste...' daremos más información sobre las envolventes de volumen.

Palabras clave asociadas: ENT, SOUND.

EOF

Sintaxis: EOF

Ejemplo:

```
10 OPENIN "ex1.bas"
20 WHILE NOT EOF
30 LINE INPUT #9,a$
40 PRINT a$
50 WEND
60 CLOSEIN
run
```

Función. Comprueba si se ha alcanzado el final del fichero (End of File). Da el valor -1 (verdadero) si no hay ningún fichero abierto o si se ha alcanzado el final del fichero; da 0 (falso) en caso contrario.

Palabras clave asociadas: OPENIN, CLOSEIN.

ERASE

Sintaxis: ERASE <lista de:<nombre de variable>>

Ejemplo:

```
DIM a(100),b$(100)
ERASE a,b$
```

Orden. Borra el contenido de una o varias listas y libera el espacio de memoria ocupado por ellas.

Palabras clave asociadas: DIM.

ERL

Sintaxis: ERL

Ejemplo:

```
10 ON ERROR GOTO 30
20 GOTO 1000
30 PRINT "Error en la línea";ERL
40 END
run
```

Función. Da el número de la línea en la que se ha detectado el último error. En el programa ejemplo, el error está en la línea 20, y éste es el número generado por ERL en la línea 30.

Palabras clave asociadas: DERR, ERR, ERROR, ON ERROR GOTO, RESUME.

ERR

Sintaxis: ERR

Ejemplo:

```
GOTO 500
Line does not exist Ready
PRINT ERR
8
```

Función. Da el número del último error detectado. En el capítulo titulado 'Para su referencia...' se da la lista de los mensajes de error. El programa ejemplo muestra que el error número 8 es 'Line does not exist' ("no existe esa línea").

Palabras clave asociadas: DERR, ERL, ERROR, ON ERROR GOTO, RESUME.

ERROR

Sintaxis: ERROR <expresión entera>

Ejemplo:

```
10 IF INKEYS="" THEN 10 ELSE ERROR 17
run
```

Orden. Invoca el error de número especificado por <expresión entera>. En el capítulo 'Para su referencia...' se da la lista de los mensajes de error 1 a 32. BASIC trata el ERROR como si efectivamente lo hubiera detectado: salta a la rutina de gestión de errores (si la hay) y genera los valores correctos de ERR y ERL.

La orden ERROR seguida de una <expresión entera> cuyo valor esté entre 33 y 255 sirve para crear mensajes de error a medida de las necesidades del programador, como ilustra el siguiente ejemplo:

```
10 ON ERROR GOTO 100
20 INPUT "Escriba una letra ";a$
```

```

30 IF LEN(A$)<>1 THEN ERROR 100
40 GOTO 20
100 IF ERR=100 THEN 110 ELSE 130
110 PRINT CHR$(7)
120 PRINT "Le dije que UNA letra!"
130 RESUME 20
run

```

Palabras clave asociadas: ERL, ERR, ON ERROR GOTO, RESUME.

EVERY

Sintaxis: EVERY <periodo de tiempo>[, <número de temporizador>]

GOSUB <número de línea>

Ejemplo:

```

10 EVERY 50,1 GOSUB 30 20 GOTO 20
30 SOUND 1,20
40 RETURN
run

```

Orden. Invoca una subrutina de BASIC a intervalos de tiempo regulares. El <periodo de tiempo> especifica el intervalo en unidades de 0.02 s (cincuentavos de segundo).

El <número del temporizador> (margen de 0 a 3) especifica cuál de los cuatro cronómetros se debe utilizar. El temporizador número 3 tiene la prioridad más alta; el número 0 (que es el implícito), la más baja.

Cada uno de los cuatro cronómetros puede tener una subrutina asociada.

En la parte 3 del capítulo 'Cuando usted guste...' daremos más amplia información acerca de las interrupciones.

Palabras clave asociadas: AFTER, REMAIN.

EXP

Sintaxis: EXP (<expresión numérica>)

Ejemplo:

```

PRINT EXP(6.876)
968.743625

```

Función. Calcula el número e elevado a la potencia dada por <expresión numérica>. El número e es la base de los logaritmos neperianos, aproximadamente igual a 2.7182818.

Palabras clave asociadas: LOG.

FILL

Sintaxis: FILL <tinta>

Ejemplo:

```

10 MODE 0
20 FOR n=1 TO 500
30 PRINT"0";
40 NEXT
50 colorpluma=2+RND*13

```

```
60 FILL colorpluma
70 GOTO 50
run
```

Orden. Rellena una región de la pantalla gráfica. Los límites de la región son rectas dibujadas con la tinta de la pluma de gráficos actual o con la tinta utilizada para rellenar (margen de 0 a 15).

El relleno se realiza partiendo de la posición actual del cursor gráfico. Si esta posición está en un límite, no se produce relleno alguno.

Palabras clave asociadas. GRAPHICS PEN.

Nota del revisor (2008): La orden FILL está disponible en los Amstrad CPC6128 y en los CPC664, pero no en el modelo inicial CPC464.

FIX

Sintaxis: FIX (<expresión numérica>)

Ejemplo:

```
PRINT FIX(9.99999)
9
```

Función. Da la parte entera de la <expresión numérica>, ignorando su parte decimal.

Palabras clave asociadas: CINT, INT, ROUND.

FN

Véase DEF FN.

FOR

Sintaxis: FOR <variable sencilla> = <principio> TO <fin> [STEP <salto>]

Ejemplo:

```
10 FOR n=2 TO 8 STEP 2
20 PRINT n;
30 NEXT n
40 PRINT ", y ocho dieciseis"
run
```

Orden. Ejecuta el grupo de instrucciones comprendidas entre FOR y NEXT un número dado de veces, incrementando el valor de la variable de control <variable del contador> en un número especificado por <salto> desde (principio) hasta <fin>. Si no se especifica ningún valor de <salto> para STEP, se supone el 1.

El <salto> puede ser una <expresión numérica>. Si su valor es negativo, el parámetro <principio> tiene que ser mayor que el parámetro <fin>, pues de no ser así la variable del contador no será modificada.

Los bucles FOR ... NEXT pueden ser 'anidados', es decir, programados uno dentro de otro.

No es necesario especificar el nombre de la variable a continuación de NEXT, ya que BASIC determina automáticamente a qué FOR corresponde cada NEXT 'anónimo'.

Palabras clave asociadas: NEXT, STEP, TO.

FRAME

Sintaxis: FRAME

Ejemplo:

```
10 MODE 0
20 PRINT "sin FRAME"
30 TAG
40 MOVE 0,200
50 FOR x=0 TO 500 STEP 4
60 IF f=1 THEN FRAME
70 MOVE x,200
80 PRINT " ";CHR$(143);
90 NEXT
100 IF f=1 THEN RUN
110 CLS
120 TAGOFF
130 PRINT "con FRAME"
140 f=1
150 GOTO 30
run
```

Orden. Sincroniza la escritura de caracteres gráficos en la pantalla con el retorno del haz en el monitor. El efecto global es hacer que el movimiento de símbolos por la pantalla sea más suave, sin parpadeo ni discontinuidad.

Palabras clave asociadas: TAG, TAGOFF.

Nota del revisor (2008): La orden FRAME está disponible en los Amstrad CPC6128 y en los CPC664, pero no en el modelo inicial CPC464. En los CPC464 se puede imitar, usando en su lugar:

```
CALL &BD19
```

FRE

Sintaxis: FRE (<expresión numérica>)

FRE (<expresión literal>)

Ejemplo:

```
PRINT FRE(0)
PRINT FRE("")
```

Función. Da el espacio libre de memoria utilizable por BASIC. En la forma FRE ("") realiza una 'limpieza de memoria' antes de dar el número de bytes libres.

Nota. BASIC solamente utiliza el primer banco de memoria, es decir, los primeros 64K.

Palabras clave asociadas: HIMEM, MEMORY.

GOSUB

Sintaxis: GOSUB <número de línea>

Ejemplo:

```
GOSUB 210
```

Orden. Invoca una subrutina de BASIC; es decir, reconduce el programa hacia el <número de línea> especificado. El final de la subrutina estará indicado por la orden RETURN; cuando se llega a ella, el programa retorna a la instrucción siguiente al GOSUB que invocó la subrutina.

Palabras clave asociadas: RETURN.

GOTO

Sintaxis: GOTO <número de línea>

Ejemplo:

```
GOTO 90
```

Orden. Hace que la ejecución del programa salte a la línea especificada.

Palabras clave asociadas: ninguna.

GRAPHICS PAPER

Sintaxis: GRAPHICS PAPER <tinta>

Ejemplo:

```
10 MODE 0
20 MASK 15
30 GRAPHICS PAPER 3
40 DRAW 640,0
run
```

Orden. Especifica la <tinta> del papel de la pantalla de gráficos, es decir, el color de fondo sobre el que se dibujan los gráficos. Cuando se dibujan líneas continuas, el papel gráfico se hace invisible. En el programa ejemplo, la orden MASK hace que la recta que se dibuja sea discontinua y que, por lo tanto, el papel gráfico se pueda ver debajo de ella.

La tinta del papel gráfico (margen de 0 a 15) se utiliza también como fondo de los caracteres que se escriben mediante TAG y para reteñir la pantalla gráfica cuando no se especifica otra tinta tras la instrucción CLG.

Palabras clave asociadas: CLG, GRAPHICS PEN, INK, MASK, TAG, TAGOFF.

GRAPHICS PEN

Sintaxis: GRAPHICS PEN [<tinta>] [, <modo de fondo>]

Ejemplo:

```
10 MODE 0
20 GRAPHICS PEN 15
30 MOVE 200,0
40 DRAW 200,400
50 MOVE 639,0
60 FILL 15
run
```

Orden. Especifica la <tinta> con la que en lo sucesivo se van a dibujar rectas y puntos. El <modo de fondo> puede ser:

- 0 Fondo opaco
- 1 Fondo transparente

(El fondo transparente afecta al papel gráfico de los caracteres escritos con TAG y a los espacios que quedan entre los trazos de las líneas discontinuas.)

Se puede omitir cualquiera de los dos parámetros, pero no ambos. Si se omite un parámetro, la característica controlada por él permanece inalterada.

Palabras clave asociadas: GRAPHICS PAPER, INK, MASK, TAG, TAGOFF.

HEX\$

Sintaxis: HEX\$ (<expresión entera> [, <anchura del campo>])

Ejemplo:

```
PRINT HEX$ (255,4)
00FF
```

Función. Produce una cadena literal que es la versión hexadecimal de la <expresión entera>. La longitud de la cadena es la especificada por el parámetro <anchura de campo> (margen de 0 a 16). Si la longitud es demasiado pequeña, la cadena resultante no se trunca, sino que consta de todos los caracteres necesarios.

El valor de la <expresión entera> tiene que estar comprendido entre -32768 y +65535.

Palabras clave asociadas: BIN\$, DEC\$, STR\$, UNT.

HIMEM

Sintaxis: HIMEM

Ejemplo:

```
PRINT HIMEM
42619
```

Función. Da la dirección de la posición de memoria más alta accesible a BASIC (la cual puede ser modificada con la orden MEMORY).

Nota. BASIC solamente utiliza el primer banco de memoria, es decir, los primeros 64K.

Palabras clave asociadas: FRE, MEMORY, SYMBOL, SYMBOL AFTER.

IF

Sintaxis: IF <expresión lógica> THEN <opción> [ELSE <opción>]

Ejemplo:

```
10 MODE 1
20 x=CINT(RND*100)
30 PRINT "Adivine mi numero (de 0 a 100)"
40 INPUT n
50 IF n<x THEN PRINT n;"es demasiado bajo
60 IF n>x THEN PRINT n;"es demasiado alto 9.0
70 IF n=x THEN 80 ELSE c=c+1:GOTO 40
80 PRINT "Muy bien. Lo ha conseguido en";
90 PRINT c+1;"intentos!"
```

run

Orden. Determina si la <expresión lógica> tiene el valor 'verdadero'. Si es así, ejecuta la primera <opción>. Si la <expresión lógica> tiene el valor 'falso', ejecuta la <opción> especificada después de ELSE; si no se ha incluido ELSE, BASIC pasa a la siguiente línea del programa.

Las órdenes IF THEN ... se pueden anidar, sin más límite que la longitud de la línea. La orden (o la serie de órdenes anidadas) termina en el final de la línea, de modo que no es posible poner en la misma línea instrucciones que sean independientes de la orden IF THEN .

Si la <opción> especificada consiste en un salto de línea, la orden puede tener cualquiera de las tres formas siguientes:

```
IF a=1 THEN 100
IF a=1 GOTO 100
IF a=1 THEN GOTO 100
```

Palabras clave asociadas: ELSE, GOTO, THEN.

INK

Sintaxis: INK <tinta>, <color> [, <color>]

Ejemplo:

```
10 MODE 1:PAPER 0:PEN 1
20 FOR p=0 TO 1
30 FOR i=0 TO 26
40 INK p,i
50 LOCATE 16,12:PRINT "ink";p;",";i
60 FOR t=1 TO 400:NEXT t,i,p
70 INK 0,1:INK 1,24:CLS
run
```

Orden. Asigna uno o dos colores a la <tinta> dada. El parámetro <tinta> es el número de referencia de la tinta (margen de 0 a 15), por el que se la cita en las órdenes PEN y PAPER. El primer <color> debe ser una expresión entera cuyo valor esté comprendido entre 0 y 26. Si se incluye el segundo <color>, la tinta alterna entre los dos colores, a una velocidad que se controla con la orden SPEED INK.

Palabras clave asociadas: GRAPHICS PAPER, GRAPHICS PEN, PAPER, PEN, SPEED INK.

INKEY

Sintaxis: INKEY (<expresión entera>)

Ejemplo:

```
10 IF INKEY(55)<>32 THEN 10
20 PRINT "Ha pulsado [SHIFT] y V"
30 CLEAR INPUT
run
```

Función. Examina el teclado para determinar si se ha pulsado una tecla determinada. La exploración se realiza una vez cada 0.02 s.

Este función es útil para averiguar si se ha pulsado cierta tecla, comprobando si el número generado por INKEY es -1 (el cual es independiente de la situación de [SHIFT] y [CTRL]).

El ejemplo anterior detecta cuándo se pulsa V (tecla número 55) con [SHIFT] y sin [CTRL]. El esquema de la numeración de las teclas está grabado en la carcasa del ordenador, a la derecha del teclado, e impreso en el capítulo titulado 'Para su referencia...!.

Los valores generados por INKEY en función de la situación de las teclas [SHIFT] y [CTRL] son los siguientes:

Valor generado	[SHIFT]	[CONTROL]	Tecla especificada
-1	Indiferente	Indiferente	No pulsada
0	No pulsada	No pulsada	Pulsada
32	Pulsada	No pulsada	Pulsada
128	No pulsada	Pulsada	Pulsada
160	Pulsada	Pulsada	Pulsada

Palabras clave asociadas: CLEAR INPUT, INKEY\$, JOY.

INKEY\$

Sintaxis: INKEY\$

Ejemplo:

```
10 CLS
20 PRINT "Elija Si o No (S/N):"
30 a$=INKEY$
40 IF a$="" THEN 30
50 IF a$="s" OR a$="S" THEN 80
60 IF a$="n" OR a$="N" THEN 90
70 GOTO 30
80 PRINT "Ha respondido SI":END
90 PRINT "Ha respondido NO"
run
```

Función. Examina el teclado para determinar si se ha pulsado alguna tecla; de ser así, genera una cadena literal consistente en el carácter correspondiente a la tecla pulsada. Si no se ha pulsado ninguna tecla, genera la cadena vacía. En el programa ejemplo, las líneas 30 y 40 forman un bucle que no deja de repetirse mientras no se pulse una tecla.

Palabras clave asociadas: CLEAR INPUT, INKEY.

INP

Sintaxis: INP (<número de puerta>)

Ejemplo:

```
PRINT INP (&FF77)
255
```

Función. Da el valor de entrada leído en la dirección de entrada/salida especificada por el parámetro <número de puerta>.

Palabras clave asociadas: OUT, WAIT.

INPUT

Sintaxis: INPUT [#<expresión de número de canal>],[;][<cadena literal entre comillas>]

<separador>] <lista de:<variable> >

Ejemplo:

```
10 MODE 1
20 INPUT "Escriba los dos factores (separados por una coma): ",a,b
30 PRINT a;"por";b;"es";a*b
40 GOTO 20
run
```

Orden. Capta una entrada de datos por el canal especificado (Por el #0, si no se especifica ninguno).

El primer signo de punto y coma [:] es opcional e inhibe la función de retorno del carro/avance de línea que de otra forma se produciría al terminar de ejecutarse la instrucción.

El <separador> tiene que ser una coma o un punto y coma. El punto y coma hace que se escriba el signo de interrogación; la coma lo impide.

Si el dato introducido es de tipo incorrecto (por ejemplo, cuando se escribe la letra O en lugar del número 0 en la captación de un dato numérico), BASIC responde con el mensaje:

?Redo from Start

y cualquier otro texto que se haya programado.

Toda introducción de datos a través del teclado debe terminar con la pulsación de la tecla [RETURN].

Palabras clave asociadas: LINE INPUT.

INSTR

Sintaxis: INSTR ([<posición de partida>,]<cadena en la que se busca>, <cadena buscada>)

Ejemplo:

```
10 CLS:FOR n=1 TO 26
20 alfabeto$=alfabeto$+CHR$(n+64)
30 NEXT
40 INPUT "Escriba una letra: ",a$
50 b$=UPPER$(a$)
60 PRINT b$;" es la letra numero";
70 PRINT INSTR(alfabeto$,b$);
80 PRINT "del alfabeto":PRINT
90 GOTO 40
run
```

Función. Determina si la <cadena buscada> está contenida en la <cadena en la que se busca>. Si lo está, da la posición del carácter de la <cadena en la que se busca> en el que se ha producido la primera coincidencia. Si no lo está, da el número 0.

El parámetro opcional <posición de partida> indica en qué lugar de la <cadena en la que se busca> debe empezar la búsqueda; debe ser una expresión entera que dé un valor comprendido entre 1 y 255.

Palabras clave asociadas: ninguna.

INT

Sintaxis: INT <expresión numérica>)

Ejemplo:

```
PRINT INT(-1.995)
-2
```

Función. Redondea el valor de la <expresión numérica> al número entero inferior, despreciando la parte decimal. Da el mismo valor que FIX cuando el argumento es positivo, pero una unidad menos que FIX cuando el argumento es negativo y no entero.

Palabras clave asociadas: CINT, FIX, ROUND.

JOY

Sintaxis: JOY (<expresión entera>)

Ejemplo:

```
10 PRINT "Para detener el programa ";
20 PRINT "accione el joystick"
30 IF JOY(0)<>0 THEN END
40 GOTO 10
run
```

Función. Lee bit a bit la situación del joystick especificado por la <expresión entera> (que debe valer 0 o 1). Genera un valor decimal según la siguiente tabla:

Bit	Decimal
0 Arriba	1
1 Abajo	2
2 Izquierda	4
3 Derecha	8
4 Disparo 2	16
5 Disparo 1	32

Así, en el programa ejemplo, si el botón principal de disparo (Disparo 2) del primer joystick está accionado mientras la empuñadura está inclinada hacia la izquierda, la función JOY(0) da el número decimal 20, que es la suma de 16 (Disparo 2) más 4 (Izquierda).

En el capítulo 'Para su referencia...' daremos más información acerca de los joysticks.

Palabras clave asociadas: CLEAR INPUT, INKEY.

KEY

Sintaxis: KEY <código expansible>, <expresión literal>

Ejemplo:

```
KEY 11,"border 13:paper 0:pen 1:ink 0,13: ink 1,0:mode 2:list"+CHR$(13)
y ahora pulse la tecla [INTRO].
```

Orden. Asigna la <expresión literal> al <código expansible> especificado. El sistema maneja 32 códigos expansibles, del 0 al 3Y; identificados por los códigos de tecla 128 a 159. Los códigos de tecla del 128 (generado por la tecla 0 del teclado numérico) al 140 (generado por la tecla [INTRO] con [CONTROL]) tienen asignadas implícitamente las funciones de escribir los dígitos del 0 al 9, el punto, [RETURN] y RUN"[RETURN] (para operación con cinta), respectivamente,

pero se les pueden asignar otras <expresiones literales> diferentes con KEY. Los códigos expansibles 13 a 31 (códigos de tecla 141 a 159) tienen asignada implícitamente la cadena vacía, pero se los puede expandir con KEY para luego asignarlos a teclas concretas con KEY DEF.

El parámetro <código expansible> incluido en la orden KEY puede estar en el margen de 0 a 31, o bien en el margen 128 a 159 para reflejar directamente códigos de tecla. (Véase la ilustración del capítulo 'Para su referencia...').

Las <expresiones literales> con las que se expandan los códigos expansibles pueden ocupar como máximo un total de 120 caracteres. Si se intenta sobrepasar ese límite se produce el error 'Improper argument' ('argumento inadecuado', error número 5).

Palabras clave asociadas: KEY DEF.

KEY DEF

Sintaxis: KEY DEF <número de tecla>, <repetir> [, <normal> [, <shift> [, <control>]]]

Ejemplo:

```
KEY 159, "esta es la tecla TAB"
KEY DEF 68,1,159
```

Después de escribir estas órdenes, pulse la tecla [TAB].

Orden. Define los códigos de tecla que van a ser generados por la tecla de número especificado. El <número de tecla> tiene que estar en el margen de 0 a 79 (véanse los números de tecla en el diagrama grabado a la derecha del teclado, o bien el esquema del capítulo 'Para su referencia...'). En la definición se pueden incluir los códigos de tecla que deben ser generados por la tecla cuando se la pulsa sola y cuando se la pulsa en combinación con [MAYS] y [CONTROL]. Todos estos parámetros son opcionales.

El parámetro <repetir> puede tener los valores 1 (para que la tecla sea de repetición) o 0 (para que no lo sea). La velocidad de repetición se puede definir con SPEED KEY.

En el ejemplo anterior, primero se asigna una cadena literal al código 159 (equivalente al código expansible 31), y luego se especifica que la tecla número 68 (la tecla [TAB]) debe ser de repetición (1) y generar el código 159 cuando se la pulse sola.

La situación normal se restaura, en el ejemplo anterior, mediante la orden:

```
KEY DEF 68,0,9
```

ya que el 9 es el código ASCII normalmente generado por [TAB].

Palabras clave asociadas: KEY, SPEED KEY.

LEFT\$

Sintaxis: LEFT\$ (<expresión literal>, <longitud deseada>)

Ejemplo:

```
10 CLS
20 a$="AMSTRAD"
30 FOR n=1 TO 7
40 PRINT LEFT$(a$,n)
50 NEXT
run
```

Función. Produce una cadena que consta del número de caracteres dado por <longitud deseada> (margen de 0 a 255) tomados de la izquierda de la <expresión literal>. Si la longitud de la <expresión literal> es menor que la <longitud deseada>, la cadena producida es la <expresión literal> completa.

Palabras clave asociadas: MID\$, RIGHT\$.

LEN

Sintaxis: LEN (<expresión literal>)

Ejemplo:

```
10 LINE INPUT "Escriba una frase: ",a$
20 PRINT "La frase tiene";
30 PRINT LEN(a$);"caracteres."
run
```

Función. Da la longitud de la <expresión literal>, es decir, el número de caracteres de que consta ésta.

Palabras clave asociadas: ninguna.

LET

Sintaxis: LET <variable> = <expresión>

Ejemplo:

```
LET x=100
```

Orden. Asigna un valor a una variable. Es una reliquia de las primeras versiones de BASIC, innecesaria en el BASIC de AMSTRAD. En el ejemplo anterior habría bastado con escribir:

```
x=100
```

Palabras clave asociadas: ninguna.

LINE INPUT

Sintaxis: LINE INPUT [#<expresión de número de canal>.] [;]

[<cadena entre comillas> <separador>] <variable literal>

Ejemplo:

```
10 LINE INPUT "Escriba una línea de texto: ",a$
20 CLS
30 PRINT "La variable a$ es igual a: "
40 PRINT a$
run
```

Orden. Acepta la introducción de una línea entera de texto por el canal especificado (o por el #0, si no se especifica ninguno). El primer signo de punto y coma [;] es opcional y, si se lo incluye, inhibe el retorno del carro/avance de línea que de otra forma se realizaría al terminar de ejecutarse la orden.

El <separador> tiene que ser una coma o un punto y coma. El punto y coma hace que se escriba el signo de interrogación; la coma lo impide.

La captación de datos a través del teclado debe terminar con la pulsación de la tecla [RETURN].

La captación de datos por el canal #9 (disco o cinta) termina cuando se recibe un carácter de retorno del carro o cuando se han recibido 255 caracteres (lo que antes ocurra).

Palabras clave asociadas: INPUT.

LIST

Sintaxis: LIST [<margen de números de línea>] [, <expresión de número de canal>]

Ejemplo:

```
LIST 100-1000, #1
```

Orden. Envía el listado de las líneas especificadas al canal especificado. El canal implícito es el #0. La impresora es el canal #8. La acción de listar se puede suspender momentáneamente pulsando [ESC] una vez, tras lo cual se la puede reanudar pulsando la barra espaciadora o abandonar definitivamente pulsando [ESC] por segunda vez.

Si se omite el primer número de <margen de números de línea>, se entiende 'desde el principio del programa hasta ...'; si se omite el segundo número, se entiende ... hasta el final del programa'. Así,

```
LIST -200
```

lista las primeras líneas del programa, hasta la 200 inclusive. En cambio,

```
LIST 50-
```

lista desde la línea 50 en adelante. Finalmente,

```
LIST
```

lista el programa completo.

Palabras clave asociadas: ninguna.

LOAD

Sintaxis: LOAD <nombre de fichero>[, <expresión de dirección de memoria>]

Ejemplo:

```
LOAD "fichero. xyz",&2AF8
```

Orden. Lee un programa de BASIC grabado en disco y lo carga en la memoria, reemplazando al actual. Si se especifica la <expresión de dirección de memoria> opcional, LOAD carga un fichero binario en esa dirección, en lugar de hacerlo en la dirección a partir de la cual se lo grabó.

Los programas de BASIC protegidos no se pueden cargar por este método, pues el ordenador los borra inmediatamente de la memoria. Esos programas deben ser cargados con RUN o CHAIN.

Palabras clave asociadas: CHAIN, CHAIN MERGE, MERGE, RUN, SAVE.

LOCATE

Sintaxis: LOCATE [#<expresión de número de canal>,]<coordenada x>, <coordenada y>

Ejemplo:

```
10 MODE 1
20 FOR n=1 TO 20
30 LOCATE n,n
40 PRINT CHR$(143);"posicion";
50 PRINT n;",";n
60 NEXT
run
```

Orden. Coloca el cursor de texto del canal indicado en la posición especificada por las coordenadas x,y. La posición 1,1 es el extremo superior izquierdo de la ventana. El canal #0 es el canal implícito.

Palabras clave asociadas: WINDOW.

LOG

Sintaxis: LOG (<expresión numérica>)

Ejemplo:

```
PRINT LOG (9999)
9.21024037
```

Función. Calcula el logaritmo natural o neperiano de la <expresión numérica>, la cual debe ser mayor que cero.

Palabras clave asociadas: EXP, LOG10.

LOG10

Sintaxis: LOG10 (<expresión numérica>)

Ejemplo:

```
PRINT LOG10(9999)
3.99995657
```

Función. Calcula el logaritmo decimal de la <expresión numérica>, la cual debe ser mayor que cero.

Palabras clave asociadas: EXP, LOG.

LOWER\$

Sintaxis: LOWER\$ (<expresión literal>)

Ejemplo:

```
10 a$="OBSERVE COMO SE CONVIERTEN LAS LETRAS A "
20 PRINT LOWER$(a$+"MINUSCULAS")
run
```

Función. Da una cadena literal que es copia de la <expresión literal> especificada, pero convirtiendo las letras mayúsculas en minúsculas y dejando inalterados los restantes caracteres. Esta función es útil en el proceso de datos que puedan contener minúsculas mezcladas con mayúsculas.

Palabras clave asociadas: UPPER\$.

MASK

Sintaxis: MASK [<expresión entera>][, <situación del primer punto>]

Ejemplo:

```
10 MODE 0:INK 5,21:INK 8,16
20 MOVE -100*RND,400*RND
30 WHILE XPOS<640
40 FOR x=1 TO 8
50 MASK 2^(8-x)
60 DRAWR 32,0,x,1:MOVER -32,0
70 NEXT
80 MOVER 34,0
90 WEND:GOTO 20
run
```

Orden. Define una 'máscara' o plantilla que se puede utilizar en el dibujo de rectas. La expresión entera> (margen de 0 a 255) determina la situación de los diversos bits de cada grupo de 8 pixels adyacentes: 0 para pixel apagado, 1 para pixel iluminado.

El parámetro <situación del primer punto> determina si el primer punto de la recta debe ser iluminado (1) o no (0).

Se puede omitir cualquiera de los dos parámetros, pero no ambos. La situación de la característica correspondiente al parámetro omitido permanece inalterada.

Palabras clave asociadas: DRAW, DRAWR, GRAPHICS PAPER, GRAPHICS PEN.

MAX

Sintaxis: MAX (<lista de:<expresión numérica> >)

Ejemplo:

```
10 n=66
20 PRINT MAX(1,n,3,6,4,3)
run
66
```

Función. Da el valor máximo de los incluidos en la <lista de:<expresión numérica> >.

Palabras clave asociadas: MIN.

MEMORY

Sintaxis: MEMORY <expresión de dirección de memoria>

Ejemplo:

```
MEMORY &20AA
```

Orden. Determina el espacio de memoria utilizable por BASIC al fijar la dirección de su límite superior.

Nota. BASIC solamente utiliza el primer banco de memoria, es decir, los primeros 64K.

Palabras clave asociadas: FRE, HIMEM, SYMBOL, SYMBOL AFTER.

MERGE

Sintaxis: MERGE <nombre del fichero>

Ejemplo:

```
MERGE "nuevopro.bas"
```

Orden. Lee un programa grabado en disco y lo superpone al actualmente almacenado en la memoria.

Las líneas del programa leído se superponen y sustituyen a las que hubiera en el antiguo con el mismo número.

Los ficheros protegidos (grabados con ,p) no pueden ser mezclados por este procedimiento con el programa actual.

Palabras clave asociadas: CHAIN, CHAIN MERGE, LOAD.

MID\$

Sintaxis: MID\$ (<expresión literal>, <posición de partida>[, <longitud de la subcadena>])

Ejemplo:

```
10 MODE 1:ZONE 3
20 a$="MASSACHUSETTS"
30 PRINT "Deletreame ";a$
40 PRINT "Vale ...":PRINT
50 FOR n=1 TO LEN(a$)
60 PRINT MID$(a$,n,1),
70 FOR t=1 TO 700:NEXT t,n
80 PRINT:PRINT
90 INPUT "Ahora escriba usted otra palabra: ",a$
100 GOTO 50
run
```

Función. Genera una cadena literal copiada de la <expresión literal>. El número de caracteres copiados es el dado por <longitud de la subcadena>; la copia empieza en el carácter especificado por <posición de partida>. Si no se especifica la <longitud de la subcadena>, se copian todos los caracteres restantes, desde <posición de partida> hacia la derecha.

Si la <posición de partida> es mayor que la longitud de la <expresión literal>, se genera la cadena vacía. El margen de <posición de partida> es de 1 a 255. El de <longitud de la subcadena> es de 0 a 255.

Palabras clave asociadas: LEFT\$, RIGHT\$.

MID\$

Sintaxis: MID\$ (<variable literal>, <posición de inserción>[, <longitud de cadena nueva>])

= <expresión de cadena nueva>

Ejemplo:

```
10 a$="Amstrad"
20 MIDS(a$,3,2)="XX"
30 PRINT a$
run
AmXXrad
```

Orden. Reemplaza un número de caracteres de la <variable literal>, dado por <longitud de cadena nueva>, con caracteres tomados de <expresión de cadena nueva>. La sustitución empieza en <posición de inserción>.

Obsérvese que cuando se utiliza MID\$ como orden, su argumento debe ser una <variable literal>; no vale, pues, una cadena literal constante tal como "hola".

Palabras clave asociadas: LEFT\$, RIGHT\$.

MIN

Sintaxis: MIN (<lista de: <expresión numérica> >)

Ejemplo:

```
PRINT MIN(3,6,2.999,8,9)
2.999
```

Función. Da el valor mínimo de los incluidos en la <lista de: <expresión numérica> >.

Palabras clave asociadas: MAX.

MOD

Sintaxis: <argumento> MOD <argumento>

Ejemplo:

```
PRINT 10 MOD 3
1
PRINT 10 MOD 5
0
```

Operador. Da el resto de la división entera del primer <argumento> por el segundo.

Palabras clave asociadas: ninguna.

MODE

Sintaxis: MODE <expresión entera>

Ejemplo:

```
10 m=m+1:IF m>2 THEN m=0
20 MODE m
30 PRINT"Este es el modo";m
40 PRINT"Pulse una tecla."
50 IF INKEY$="" THEN 50 ELSE 10
run
```

Orden. Cambia el modo de pantalla (0, 1 o 2) y la borra tiéndola con la tinta número 0 (que puede no ser la tinta actualmente asignada al papel). Todas las pantallas de texto y gráficas se restauran (coincidentes con la pantalla completa); los cursores de texto y gráfico quedan en sus orígenes respectivos.

Palabras clave asociadas: ORIGIN, WINDOW.

MOVE

Sintaxis: MOVE <coordenada x>, <coordenada y>[, [<tinta>][, <modo de tinta>]]

Ejemplo:

```
10 MODE 1: TAG
20 x=RND*800-100:y=RND*430
30 MOVE x,y
40 PRINT "Estoy aqui";
50 GOTO 20
run
```

Orden. Lleva el cursor gráfico a la posición absoluta especificada por las coordenadas x,y. Se puede utilizar el parámetro opcional <tinta> para cambiar la tinta de la pluma gráfica (margen 0 a 15).

El parámetro opcional <modo de tinta> determina la futura forma de interacción entre la tinta con la que se dibuje y la ya existente en la pantalla. Los cuatro modos son:

```
0 Normal
1 XOR (OR exclusivo)
2 AND
3 OR
```

Palabras clave asociadas: MOVER, ORIGIN, XPOS, YPOS.

MOVER

Sintaxis: MOVER <incremento de x> , <incremento de y>[, [<tinta>][, <modo de tinta>]]

Ejemplo:

```
10 MODE 1:TAG:MOVE 0,16
20 PRINT "primero se sube";
30 FOR n=1 TO 8
40 MOVER -56,16
50 PRINT "sube";:NEXT:PRINT " y luego se baja";
60 FOR n=1 TO 8
70 MOVER -56,-16
80 PRINT "baja";:NEXT
run
```

Orden. Lleva el cursor gráfico a un punto cuya posición, referida a la actual, viene dada por <incremento de x> e <incremento de y>. Se puede utilizar el parámetro opcional <tinta> para cambiar la tinta de la pluma gráfica (margen de 0 a 15).

El parámetro opcional (modo de tinta) determina la futura forma de interacción entre la tinta con la que se dibuje y la ya existente en la pantalla. Los cuatro modos son:

```
0 Normal
1 XOR (OR exclusivo)
2 AND
3 OR
```

Palabras clave asociadas: MOVE, ORIGIN, XPOS, YPOS.

NEW

Sintaxis: NEW

Ejemplo:

```
NEW
```

Orden. Borra el programa de BASIC actualmente residente en la memoria. No se pierden las definiciones de teclas ni se modifican las características de la pantalla (MODE, PEN, PAPER, INK, etc.) ni se borra la pantalla.

Palabras clave asociadas: ninguna.

NEXT

Sintaxis: NEXT [<lista de:<variable>>]

Ejemplo:

```
10 FOR a=1 TO 3
20 FOR b=0 TO 26
30 MODE 1
40 PEN a:BORDER b
50 PRINT "pen";a;" border";b
60 FOR c=1 TO 500
70 NEXT c,b,a
run
```

Orden. Señala el final del bucle FOR ... NEXT. La orden NEXT puede ser anónima o mencionar el nombre de la variable del FOR correspondiente. Obsérvese en el ejemplo anterior que las variables de la lista deben estar en orden inverso al de sus correspondientes órdenes FOR, de modo que los bucles no queden solapados, sino anidados.

Palabras clave asociadas: FOR, STEP, TO.

NOT

Sintaxis: NOT <argumento>

Ejemplo:

```
IF NOT THEN "juan"<"pepe" PRINT "verdadero" ELSE PRINT "falso"
falso
IF NOT "gato">"perro" THEN PRINT "verdadero" ELSE PRINT "falso"
verdadero
PRINT NOT -1
0
PRINT NOT 0
-1
```

Operador. Realiza la negación lógica sobre enteros invirtiendo todos los bits del argumento.

En la parte 2 el capítulo 'Cuando usted guste...' daremos más amplia información sobre las operaciones lógicas.

Palabras clave asociadas: AND, OR, XOR.

ON BREAK CONT

Sintaxis: ON BREAK CONT

Ejemplo:

```
10 ON BREAK CONT
20 PRINT "El programa CONTinua aunque usted intente interrumpirlo con
   [ESC]":PRINT
30 FOR t=1 TO 1000:NEXT:GOTO 20
run
```

Orden. Inhibe la posibilidad de interrumpir el programa pulsando la tecla [ESC], pues intercepta la interrupción y continúa la ejecución. Esta orden se debe utilizar con precaución, ya que el programa no podrá ser detenido más que reiniciando la máquina; grabe el programa antes de ejecutarlo.

Dentro de un programa, esta orden se anula con ON BREAK STOP.

Palabras clave asociadas: ON BREAK GOSUB, ON BREAK STOP.

ON BREAK GOSUB

Sintaxis: ON BREAK GOSUB <número de línea>

Ejemplo:

```
10 ON BREAK GOSUB 40
20 PRINT "Programa funcionando"
30 GOTO 20
40 CLS:PRINT "Pulsando dos veces
50 PRINT"se invoca la subrutina"
60 FOR t=1 TO 2000:NEXT
70 RETURN
run
```

Orden. Provoca el salto a la subrutina especificada por <número de línea> cuando se pulsa dos veces la tecla [ESC].

Palabras clave asociadas: ON BREAK CONT, ON BREAK STOP, RETURN.

ON BREAK STOP

Sintaxis: ON BREAK STOP

Ejemplo:

```
10 ON BREAK GOSUB 40
20 PRINT "Programa funcionando"
30 GOTO 20
40 CLS:PRINT "Pulsando dos veces
50 PRINT"se invoca la subrutina"
60 FOR t=1 TO 2000:NEXT
65 ON BREAK STOP
70 RETURN
run
```

Orden. Anula las órdenes ON BREAK CONT y ON BREAK GOSUB, permitiendo así que el programa pueda ser interrumpido normalmente con la tecla [ESC]. En el programa ejemplo, la

orden ON BREAK GOSUB sólo actúa una vez, pues la línea 65 de la rutina de interrupción la anula.

Palabras clave asociadas: ON BREAK CONT, ON BREAK GOSUB.

ON ERROR GOTO

Sintaxis: ON ERROR GOTO <número de línea>

Ejemplo:

```
10 ON ERROR GOTO 60
20 CLS:PRINT "Si detecto un error, ";
30 PRINT "listare el programa"
40 FOR t=1 TO 4000:NEXT
50 GOTO 100
60 PRINT "Hay un error en la línea";
70 PRINT ERL:PRINT:LIST
run
```

Orden. Provoca el salto a la línea especificada cuando se detecta un error en el programa.

La forma ON ERROR GOTO 0 desactiva la gestión de errores programada por el usuario y activa la normal de BASIC.

Véase también la orden RESUME.

Palabras clave asociadas: DERR, ERL, ERR, ERROR, RESUME.

ON <expresión> GOSUB

Sintaxis: ON <selector> GOSUB <lista de: <número de línea>>

Ejemplo:

```
10 PAPER 0:PEN 1:INK 0,1
20 CLS:PRINT "MENU DE OPCIONES":PRINT
30 PRINT "1. Cambiar borde":PRINT
40 PRINT "2. Cambiar pluma":PRINT
50 PRINT "3. Cambiar modo":PRINT
60 INPUT "Elija del 1 al 3: ",x
70 ON x GOSUB 90,110,130
80 GOTO 20
90 b=b-1:IF b<0 THEN b=26
100 BORDER b:RETURN
110 p=p-1:IF p<2 THEN p=26
120 INK 1,p:RETURN
130 m=m-1:IF m<0 THEN m=2
140 MODE m:RETURN
run
```

Orden. Determina, en función del valor de <selector>, qué subrutina de BASIC debe ser invocada. El margen de valores de <selector> es de 0 a 255. La rutina elegida es la que se encuentra en la lista en el lugar indicado por <selector>. Así, en el ejemplo anterior, si x=1, se invoca la rutina de la línea 90; si x=2, la línea 110; y si x=3, la de la línea 130.

Si el valor de <selector> es cero o mayor que el número de líneas citadas en la <lista de números de línea>, no se invoca ninguna subrutina.

Palabras clave asociadas: RETURN.

ON <expresión> GOTO

Sintaxis: ON <selector> GOTO <lista de: <número de línea> >

Ejemplo:

```
10 CLS:PRINT "MENÚ DE OPCIONES":PRINT
20 PRINT "1. Listar programa":PRINT
30 PRINT "2. Editar y ampliar":PRINT
40 PRINT "3. Catalogo del disco":PRINT
50 INPUT "Elija del 1 al 3: ",x
60 ON x GOTO 80,90,100
70 GOTO 10
80 LIST
90 AUTO
100 CAT
run
```

Orden. Determina, en función del valor de <selector>, a qué línea del programa debe saltar la ejecución. El margen de valores de <selector> es de 0 a 255. La línea elegida es la que se encuentra en la lista en el lugar indicado por <selector>. Así, en el ejemplo anterior, si x=1 se salta a la línea 80; si x=2, a la línea 90; y si x=3, a la línea 100.

Si el valor de <selector> es cero o mayor que el número de líneas citadas en la <lista de números de línea>, no se produce el salto.

Palabras clave asociadas: ON expresión GOSUB.

ON SQ GOSUB

Sintaxis: ON SQ (<canal>) GOSUB <número de línea>

Ejemplo:

```
10 ENV 1,15,-1,1
20 ON SQ(1) GOSUB 60
30 MODE 0:ORIGIN 0,0,200,440,100,300
40 FOR x=1 TO 13:FRAME:MOVE 330,200,x
50 FILL x:NEXT:GOTO 40
60 READ s:IF s=0 THEN RESTORE:GOTO 60
70 SOUND 1,s,25,15,1
80 ON SQ(1) GOSUB 60:RETURN
90 DATA 50,60,90,100,35,200,24,500,0
run
```

Orden. Invoca la subrutina especificada cuando hay espacio libre en la cola de sonidos. El parámetro <canal> deber ser una expresión entera que dé uno de los siguientes valores:

- 1 para el canal A
- 2 para el canal B
- 4 para el canal C

En la parte 2 del capítulo 'Cuando usted guste...' daremos más información sobre los sonidos.

Palabras clave asociadas: RETURN, SOUND, SQ.

OPENIN

Sintaxis: OPENIN <nombre de fichero>

Ejemplo:

```
10 REM abrir un fichero del disco para entrada
20 OPENIN "datos":INPUT #9,a,a$
30 CLOSEIN:PRINT "Los dos valores son:"
40 PRINT:PRINT a,a$
run
```

Orden. Abre, en dirección de entrada, un fichero grabado en disco para hacerlo accesible al programa. El fichero tiene que ser de tipo ASCII.

El programa anterior sólo funciona si se ha creado previamente el fichero (por ejemplo, con el programa siguiente).

Palabras clave asociadas: CLOSEIN, EOF, OPENOUT.

OPENOUT

Sintaxis: OPENOUT <nombre del fichero>

Ejemplo:

```
10 REM abrir un fichero en el disco para salida
20 INPUT "Escriba un numero: ",a
30 INPUT "Escriba una palabra: ",a$
40 OPENOUT "datos"
50 WRITE #9,a,a$
60 CLOSEOUT:PRINT "Datos grabados en disco"
run
```

Orden. Abre, en dirección de salida, un fichero para su grabación en disco.

Palabras clave asociadas: OPENIN, CLOSEOUT.

OR

Sintaxis: <argumento> OR <argumento>

Ejemplo:

```
IF "juan"<"pepe" OR "perro">"gato" THEN PRINT "verdadero" ELSE PRINT "falso"
verdadero
IF "pepe"<"juan" OR "gato">"perro" THEN PRINT "verdadero" ELSE PRINT "falso"
falso
IF "juan"<"pepe" OR "gato">"perro" THEN PRINT "verdadero" ELSE PRINT "falso"
Verdadero
...
PRINT 1 OR 1
1
PRINT 0 OR 0
0
PRINT 1 OR 0
1
```

Operador. Realiza la operación lógica 'o' bit a bit entre enteros. El resultado es 1 a menos que

ambos bits sean 0.

Para más amplia información sobre operaciones lógicas, consulte la parte 2 del capítulo titulado 'Cuando usted guste ...

Palabras clave asociadas: AND, NOT, XOR.

ORIGIN

Sintaxis: ORIGIN <x> , <y> [, <izquierda>, <derecha>, <arriba>, <abajo>]

Ejemplo:

```
10 MODE 1:BORDER 13:TAG
20 ORIGIN 0,0,100,540,300,100
30 GRAPHICS PAPER 3:CLG
40 FOR x=550 TO -340 STEP -10
50 MOVE x,206
60 PRINT "Esta es una ventana grafica ";
70 FRAME:NEXT:GOTO 40
run
```

Orden. Traslada el origen de gráficos (nuevo punto 0,0) a la posición especificada por <x> e <y>.

Al mismo tiempo se pueden especificar las dimensiones de una ventana gráfica dando los cuatro últimos parámetros (opcionales). Si estos parámetros definen posiciones externas a la pantalla, el borde de la ventana será el borde de la pantalla.

Palabras clave asociadas: CLG.

OUT

Sintaxis: OUT <número de puerta>, <expresión entera>

Ejemplo:

```
OUT &F8F4,&FF
```

Orden. Envía el valor de <expresión entera> (margen de 0 a 255) a la dirección especificada por <número de puerta> .

Esta orden no debe ser utilizada si no se sabe cuáles son sus efectos.

Palabras clave asociadas: INP, WAIT.

PAPER

Sintaxis: PAPER [<expresión de número de canal>,] <tinta>

Ejemplo:

```
10 MODE 0:PEN 0:INK 0,13
20 FOR p=1 TO 15
30 PAPER p:CLS
40 LOCATE 7,12:PRINT "paper";p
50 FOR t=1 TO 500:NEXT t,p
run
```

Orden. Establece qué tinta se va a utilizar para escribir el fondo (papel) de los caracteres. Si los

caracteres se envían a la pantalla de texto, la celda se rellena con el color actual del papel antes de escribir el carácter (a menos que se haya elegido el modo transparente).

Si se omite la <expresión de número de canal> se supone el #0.

El número de tintas utilizables simultáneamente depende del modo de pantalla.

Palabras clave asociadas: GRAPHICS PAPER, INK, PEN.

PEEK

Sintaxis: PEEK (<expresión de dirección de memoria>)

Ejemplo:

```
10 MODE 1:ZONE 7
20 WINDOW 1,40,1,3:WINDOW #1,1,40,4,25
30 LOCATE 13,1:PRINT"MEMORIA"
40 PRINT:PRINT"Direccion"
50 LOCATE 20,3:PRINT"Contenido"
60 FOR n=0 TO 65535
70 p=PEEK(n)
80 PRINT #1,n,"(&;HEX$(n);)";
90 PRINT #1,TAB(20);p,"(&;HEX$(p);)"
100 NEXT
run
```

Función. Da el contenido de la posición de memoria especificada por el argumento, el cual debe estar en el margen de &0000 a &FFFF (0 a 65535). En todos los casos, el valor producido será el contenido de la posición de memoria de la RAM (no de la ROM) y estará entre &00 y &FF (entre 0 y 255).

Palabras clave asociadas: POKE.

PEN

Sintaxis: PEN [#<expresión de número de canal>],[<tinta>],[<modo de fondo>]

Ejemplo:

```
10 MODE 0:PAPER 0:INK 0,13
20 FOR p=1 TO 15
30 PEN p:PRINT SPACE$(47);"pen";p
40 FOR t=1 TO 500:NEXT t,p:GOTO 20
run
```

Orden. Establece la <tinta> (margen de 0 a 15) con la que se escribirán los caracteres en lo sucesivo en el canal especificado (en el #0 si no se especifica ninguno). El parámetro <modo de fondo> puede ser 1 (transparente) o 0 (opaco).

Se puede omitir cualquiera de los últimos parámetros, pero no ambos. La característica correspondiente al parámetro omitido permanece inalterada.

Palabras clave asociadas: GRAPHICS PEN, INK, PAPER.

PI

Sintaxis: PI

Ejemplo:

```
PRINT PI
3.14159265
```

Función. Da el valor del número π , relación de la longitud al diámetro de la circunferencia.

Palabras clave asociadas: DEG, RAD.

PLOT

Sintaxis: PLOT <coordenada x>, <coordenada y>[, [<tinta>][, <modo de tinta>]]

Ejemplo:

```
10 MODE 1:BORDER 0:PAPER 0:PEN 1
20 INK 0,0:INK 1,26:INK 2,13,26:DEG
30 FOR x=1 TO 360:ORIGIN 320,200
40 DRAW 50*COS(x),50*SIN(x),1
50 PLOT 100*COS(x),25*SIN(x):NEXT
60 ORIGIN 0,0:t=TIME+700:WHILE TIME<t
70 PLOT RND*640,RND*400:WEND
80 PLOT RND*640,RND*400,2
90 GOTO 90
run
```

Orden. Dibuja un punto en la pantalla gráfica, en la posición absoluta especificada por los parámetros <coordenada x> y <coordenada y>. También se puede especificar la <tinta> con la que se debe dibujar (margen de 0 a 15).

El parámetro opcional <modo de tinta> determina la forma de interacción entre la tinta con la que se dibuja y la ya presente en la pantalla. Los cuatro modos son:

```
0 Normal
1 XOR (OR exclusivo)
2 AND
3 OR
```

Palabras clave asociadas: GRAPHICS PEN, PLOTTR.

PLOTTR

Sintaxis: PLOTTR <incremento de x>, <incremento de y>[, [<tinta>, <modo de tinta>]]

Ejemplo:

```
10 REM dibujo de rectas con las teclas del cursor
20 BORDER 0:GRAPHICS PEN 1
30 MODE 1:PLOT 320,200
40 IF INKEY(0)=0 THEN PLOTTR 0,1
50 IF INKEY(1)=0 THEN PLOTTR 1,0
60 IF INKEY(2)=0 THEN PLOTTR 0,-1
70 IF INKEY(8)=0 THEN PLOTTR -1,0
80 IF INKEY(9)=0 THEN 30:REM copy = borrar
90 GOTO 40
run
```

Orden. Dibuja un punto en la pantalla gráfica en una posición que, referida a la actual, viene dada por los parámetros <incremento de x> e <incremento de y>. Se puede utilizar el parámetro opcional <tinta> para cambiar la tinta de la pluma gráfica (margen de 0 a 15).

El parámetro opcional <modo de tinta> determina la futura forma de interacción entre la tinta con la que se dibuja y la ya presente en la pantalla. Los cuatro modos son:

- 0 Normal
- 1 XOR (OR exclusivo)
- 2 AND
- 3 OR

Palabras clave asociadas: GRAPHICS PEN, PLOT.

POKE

Sintaxis: POKE <expresión de dirección de memoria>, <expresión entera>

Ejemplo:

```
10 FOR m=49152 TO 65535
20 POKE m,100
30 NEXT
run
```

Orden. Escribe el valor de la <expresión entera> (margen de 0 a 255) en la memoria de la máquina (RAM), en la posición especificada por el parámetro <expresión de dirección de memoria>.

Esta orden no se debe utilizar si no se sabe cuáles son sus efectos.

Palabras clave asociadas: PEEK.

POS

Sintaxis: POS (#<expresión de número de canal>)

Ejemplo:

```
10 MODE 1:BORDER 0:LOCATE 8,2
20 PRINT "Utilice las teclas de cursor a la izquierda y cursor a la derecha"
30 WINDOW 1,40,12,12:Cursors 1,1
40 FOR n=1 TO 19:PRINT CHR$(9);:NEXT
50 IF INKEY(1)<>-1 THEN PRINT CHR$(9);
60 IF INKEY(8)<>-1 THEN PRINT CHR$(8);
70 LOCATE #1,1,24
80 PRINT #1,"Posicion horizontal del ";
90 PRINT #1,"cursor de texto =";
100 PRINT #1,POS(#0):GOTO 50
run
```

Función. Da la posición horizontal del cursor de texto referida al borde izquierdo de la ventana. El parámetro <expresión de número de canal> tiene que ser especificado, aunque sea el #0.

POS(#8) da la posición horizontal de la cabeza impresora, siendo 1 la posición del extremo izquierdo.

POS(#9) da la posición lógica en el canal del disco, esto es, el número de caracteres tipográficos enviados al disco después del último retorno del carro.

Palabras clave asociadas: VPOS, WINDOW.

PRINT

Sintaxis: PRINT [<expresión de número de canal>,][<lista de:<elemento que se escribe>>]

Ejemplo:

```
10 a$="corta"
20 b$="esta cadena es mas larga"
30 PRINT a$;a$
40 PRINT a$,a$
50 PRINT
60 PRINT b$;b$
70 PRINT b$,b$
run
```

Orden. Envía los <elementos que se escriben> al canal especificado (o al #0, si no se especifica ninguno).

Cuando se utiliza un signo de punto y coma (;) para separar los elementos de la lista, el ordenador escribe cada elemento inmediatamente después del anterior; no obstante,

BASIC comprueba si el elemento siguiente cabe en la misma línea y, si no cabe, salta a la línea siguiente a pesar del signo de punto y coma.

Análogamente, la coma (,) especifica que el siguiente elemento se debe escribir en la siguiente zona de escritura; no obstante, BASIC comprueba si con el elemento actual se ha rebasado la anchura de la zona de escritura y, si es así, escribe el siguiente elemento en una zona más a la derecha.

PRINT SPC

PRINT TAB

Sintaxis:

PRINT [<expresión de número de canal>,][<lista de:<elemento que se escribe>>] [;]

[SPC <expresión entera>][<lista de:<elemento que se escribe>>]

PRINT [<expresión de número de canal>,][<lista de:<elemento que se escribe>>] [;]

[TAB (<expresión entera>) [<lista de:<elemento que se escribe>>]

Ejemplo:

```
10 PRINT "Esta es la funcion SPC"
20 FOR x=6 TO 15
30 PRINT SPC(5)"a" ;SPC(x)"b"
40 NEXT
50 PRINT "Esta es la funcion TAB"
60 FOR x=6 TO 15
70 PRINT TAB(5)"a" ;TAB(x)"b"
80 NEXT
run
```

SPC escribe el número de espacios especificado por la <expresión entera> y luego, inmediatamente a su derecha, el siguiente <elemento> (en el supuesto de que quepa en la misma línea). Por consiguiente, no es necesario poner el signo de punto y coma después de SPC.

TAB avanza (escribiendo espacios) hasta la posición especificada por la <expresión entera> ,

referida al borde izquierdo de la ventana, y luego, inmediatamente a la derecha de esa posición, escribe el siguiente <elemento> (en el supuesto de que quepa en la misma línea). Por consiguiente, no es necesario poner el signo de punto y coma después de TAB. Si la posición actual es mayor que la especificada, primero se realiza un retorno de carro y avance de línea y después el avance de los espacios necesarios.

PRINT USING

Sintaxis: PRINT [<expresión de número de canal>,]

[<lista de:<elemento que se escribe>>] [;]

[USING <plantilla de formato>][<separador><expresión>]

Ejemplo:

```
10 FOR x=1 TO 10
20 n=100000*(RND^5)
30 PRINT "Precio";USING "#####,.##";n
40 NEXT
run
```

PRINT USING permite especificar el formato de la expresión escrita con PRINT. Esto se consigue especificando una <plantilla de formato> a la que la escritura ha de amoldarse. El <separador> es una coma o un punto y coma. La <plantilla de formato> es una cadena literal construida con los siguientes 'especificadores de formato':

Formatos numéricos

Para el número en sí:

- # Cada # especifica una posición (para un dígito). Ejemplo: #####
- . Especifica la posición del punto decimal. Ejemplo: #####.##
- , (Especifica una posición.) Sólo puede aparecer antes del punto decimal. Especifica que los dígitos que queden a la izquierda del punto decimal deben separarse mediante comas en grupos de tres. Ejemplo: #####,.##

Para antes y después del número:

- PtPt (Especifica dos posiciones.) Especifica que se debe escribir un signo Pt inmediatamente antes del primer dígito o del punto decimal, después del signo + o - (si lo hay). Obsérvese que el signo Pt ocupará una de las posiciones para dígitos. Ejemplo: PtPt#####,.##
- ** (Especifica dos posiciones.) Especifica que los espacios libres a la izquierda deben ser rellenados con asteriscos. Ejemplo: **#####,.##
- **Pt (Especifica tres posiciones.) Actúa como las opciones ** y PtPt combinadas, es decir, con asteriscos por la izquierda y el signo Pt. Ejemplo: **Pt#####,.##
- \$\$ (Especifica dos posiciones.) Especifica que se debe escribir un signo \$ inmediatamente antes del primer dígito o del punto decimal, después del signo + o - (si lo hay). Obsérvese que el signo \$ ocupará una de las posiciones para dígitos. Ejemplo: \$#####,.##
- **\$ (Especifica tres posiciones.) Actúa como las opciones ** y \$\$ combinadas, es decir, con asteriscos por la izquierda y el signo \$. Ejemplo: **\$#####,.##

- + Especifica que se debe escribir el signo + o el - (según corresponda). Si se incluye el signo + al principio de la plantilla, el signo se escribirá inmediatamente antes del número (y del signo Pt o \$, si se lo ha especificado). Si se incluye el signo + al final de la plantilla, el signo se escribe a la derecha del número (y del exponente, si lo hay). Ejemplo: +####.####
- - Este signo sólo se puede poner al final de la plantilla. Especifica que, si el número es negativo, se debe escribir el signo - a su derecha (y a la derecha del exponente, si lo hay). Si el número es positivo, a su derecha se escribe un espacio. Si no se especifica esto, se escribe automáticamente un signo - a la izquierda de todos los números negativos. Ejemplo: ####.####-
- ^ ^ ^ ^ Especifica que el número se debe escribir en forma exponencial. Los signos ↑↑↑↑ se deben poner a la derecha de las posiciones para dígitos, pero a la izquierda de los signos + y - (si los hay). Ejemplo: ##.#### ^ ^ ^ ^ +

La <plantilla de formato> para números no puede contener más de 20 caracteres. Los números son redondeados antes de escribirlos con los dígitos especificados.

Si el formato es demasiado pequeño para el número que se ha de escribir, por ejemplo

```
PRINT USING "####";12345678
```

el número no se trunca, sino que se lo escribe entero, precedido de un signo % para indicar el 'fallo de formato'.

Formatos literales

```
10 CLS:a$="abcdefghijklmnopq"
20 PRINT "expresion de entrada = ";a$
30 PRINT:PRINT "con especificador ! = ";
40 PRINT USING "!";a$
50 PRINT:PRINT "con especificador \espacios\ =
60 PRINT USING "\ \";a$
70 PRINT:PRINT "con especificador & = ";
80 PRINT USING "&";a$
90 GOTO 90
run
```

- ! Especifica que solamente se debe escribir el primer carácter de la cadena. Ejemplo: !
- \ <espacios> \ Especifica que solamente se deben escribir los x primeros caracteres de la cadena, siendo x la longitud de la plantilla (incluidas las barras inclinadas hacia la izquierda). Ejemplo: \ \
- & Especifica que se debe escribir la cadena completa sin modificarla. Ejemplo: &

La <plantilla de formato> para cadenas literales no puede contener más de 255 caracteres.

Las plantillas de formato, tanto si son numéricas como si son literales, pueden ser representadas por variables literales; por ejemplo:

```
10 a$="PtPt#####,.##"
20 b$="!"
30 PRINT USING a$;12345.6789
40 PRINT USING b$;"centimos"
```

run

En la parte 2 del capítulo 'Cuando usted guste ...' daremos más información acerca de los formatos de impresión.

Palabras clave relacionadas: SPC, TAB, USING, ZONE.

RAD

Sintaxis: RAD

Ejemplo:

```
RAD
```

Orden. Hace que BASIC entienda los argumentos de las funciones trigonométricas en radianes. Ésta es la situación implícita que se establece en el momento de encender la máquina, cuando se la reinicializa y cuando se ejecutan las órdenes NEW, CLEAR, LOAD, RUN, etc.

Palabras clave asociadas: ATN, COS, DEG, SIN, TAN.

RANDOMIZE

Sintaxis: RANDOMIZE <expresión numérica>

Ejemplo:

```
RANDOMIZE 123.456  
PRINT RND  
0.258852139
```

Orden. El generador de números aleatorios de BASIC produce una sucesión pseudo-aleatoria en la que cada número depende del anterior. La sucesión es siempre la misma. RANDOMIZE hace que los números aleatorios se tomen de esa sucesión a partir de la posición dada por la <expresión numérica> o, si se omite ésta, por un valor que el usuario debe teclear.

RANDOMIZE TIME produce una sucesión que es muy difícil de repetir.

Palabras clave asociadas: RND.

READ

Sintaxis: READ <lista de: <variable> >

Ejemplo:

```
10 FOR n=1 TO 8  
20 READ a$,c  
30 PRINT a$;" ";;SOUND 1,c:NEXT  
40 DATA en,478,todas,426,las,379,escalas,358  
50 DATA musicales,319,hay,284,8,253,notas,239  
run
```

Orden. Lee datos de las listas precedidas de DATA y los asigna a variables, actualizando automáticamente el 'puntero' de forma que éste señale el siguiente dato. Se puede utilizar la orden RESTORE para hacer que el puntero señale el primer dato de una determinada lista DATA.

En la parte 2 del capítulo titulado 'Cuando usted guste...' daremos más amplia información a este

respecto.

Palabras clave asociadas: DATA, RESTORE.

RELEASE

Sintaxis: RELEASE <canales de sonido>

Ejemplo:

```
10 SOUND 65,1000,100
20 PRINT "Pulse R para liberar el sonido"
30 IF INKEY(50)>=-1 THEN 30
40 RELEASE 1
RUN
```

Orden. Libera los canales de sonido que han sido retenidos en una orden SOUND.

El parámetro <canales de sonido> tiene que dar un valor entero comprendido entre 1 y 7. Su significado es el siguiente:

- 1 libera el canal A
- 2 libera el canal B
- 3 libera los canales A y B
- 4 libera el canal C
- 5 libera los canales A y C
- 6 libera los canales B y C
- 7 libera los canales A, B y C

En la parte 2 del capítulo 'Cuando usted guste...' daremos más amplia información acerca de los sonidos.

Palabras clave asociadas: SOUND.

REM

Sintaxis: REM <resto de la línea>

Ejemplo:

```
10 REM Caza a muerte de los invasores del hiper-espacio intergalactico, por
   AMSOFT
20 REM Copyright AMSOFT 1985
```

Orden. Precede a las observaciones o anotaciones al programa. Hace que BASIC ignore el resto de la línea, en el cual pueden figurar caracteres cualesquiera, incluso el signo de dos puntos (:), que normalmente se utiliza para separar instrucciones.

En lugar de :REM se puede poner una comilla sencilla (') en todos los casos salvo en las líneas de DATA, en las que el signo ' se interpreta como integrante de una cadena literal.

Palabras clave asociadas: ninguna.

REMAIN

Sintaxis: REMAIN (<número de temporizador>)

Ejemplo:

```
10 AFTER 500,1 GOSUB 40
20 AFTER 100,2 GOSUB 50
30 PRINT "Programa funcionando":GOTO 30
40 REM esta subrutina no sera invocada porque lo impide la línea 80
50 PRINT:PRINT "El temporizador 1 va a ser ";
60 PRINT "inhibido por REMAIN."
70 PRINT "Unidades de tiempo que quedaban:";
80 PRINT REMAIN(1)
run
```

Función. Da la cuenta restante en el temporizador especificado (margen de 0 a 3) y lo desactiva.

En la parte 2 del capítulo 'Cuando usted guste...' daremos más información acerca de las interrupciones.

Palabras clave asociadas: AFTER, DI, EI, EVERY.

RENUM

Sintaxis: RENUM [<nuevo número de línea>][,<antiguo número de línea>][,<incremento>]]

Ejemplo:

```
10 CLS
20 REM esta linea sera la 123
30 REM esta linea sera la 124
40 REM esta linea sera la 125
RENUM 123,20,1
LIST
```

Orden. Renumerar las líneas del programa.

El parámetro <antiguo número de línea> especifica el número de línea actual en el cual debe comenzar la renumeración. Si se omite ese parámetro, la renumeración se realiza desde el principio del programa.

El parámetro <nuevo número de línea> especifica qué número se va a dar a la primera línea renumerada. Si se omite este parámetro, el programa renumerado empieza en la línea 10.

El parámetro <incremento> especifica el salto que habrá entre cada dos números de línea sucesivos. Si se lo omite, el incremento es 10.

RENUM modifica correctamente los números de línea citados en las instrucciones GOTO, GOSUB y similares. Sin embargo, no se modifican las referencias a números de línea que estén contenidas en expresiones literales, tales como las que intervienen en las órdenes KEY, ni tampoco las contenidas en líneas REM, ni las <expresiones de número de línea> incluidas en CHAIN y CHAIN MERGE.

Los números de línea válidos son los comprendidos entre 1 y 65535.

Palabras clave asociadas: DELETE, LIST.

RESTORE

Sintaxis: RESTORE [<número de línea>]

Ejemplo:

```

10 READ a$:PRINT a$;" ";
20 RESTORE 50
30 FOR t=1 TO 500:NEXT:GOTO 10
40 DATA restaure los datos para leerlos una vez
50 DATA y otra
run

```

Orden. Hace que el 'puntero' de datos 'apunte' hacia el principio de la línea de DATA especificada por el parámetro opcional <número de línea>. Si se omite el parámetro, el puntero señala la primera línea de DATA del programa.

En la parte 2 del capítulo 'Cuando usted guste ...' daremos más información a este respecto.

Palabras clave asociadas: DATA, READ.

RESUME

Sintaxis: RESUME [<número de línea>]

Ejemplo:

```

10 ON ERROR GOTO 60
20 FOR x=10 TO 0 STEP -1:PRINT 1/x:NEXT
30 END
40 PRINT "llegamos hasta aqui despues del error"
50 END
60 PRINT "error numero";ERR;"en la línea";ERL
70 RESUME 40
run

```

Orden. Reanuda la ejecución normal del programa una vez detectado un error y procesado por una orden ON ERROR GOTO. Si se omite el <número de línea>, el programa continúa en la misma línea en la que se produjo el error. Suprima el <número de línea> en el ejemplo anterior y ejecute nuevamente el programa:

```

70 RESUME
run

```

Palabras clave asociadas: DERR, ERL, ERR, ERROR, ON ERROR GOTO, RESUME NEXT.

RESUME NEXT

Sintaxis: RESUME NEXT

Ejemplo:

```

10 ON ERROR GOTO 90
20 PRINT "Pulse siempre [RETURN]"
30 INPUT "1";a
40 INPUT "2";a
50 input "3";a REM error de sintaxis
60 INPUT "4";a
70 INPUT "5";a
80 END
90 PRINT "error numero";ERR;"en la línea";ERL
100 RESUME NEXT
run

```

Orden. Reanuda la ejecución normal del programa una vez detectado un error y procesado por una orden ON ERROR GOTO.

RESUME NEXT reanuda la ejecución en la línea siguiente a aquélla en la que se detectó el error.

Palabras clave asociadas: DERR, ERL, ERR, ERROR, ON ERROR GOTO, RESUME.

RETURN

Sintaxis: RETURN

Ejemplo:

```
10 GOSUB 50:PRINT "despues del GOSUB":END
50 FOR n=1 TO 20
60 PRINT "subrutina"
70 NEXT:PRINT
80 RETURN
run
```

Orden. Señala el final de una subrutina. BASIC retorna de la subrutina a la instrucción inmediatamente posterior a aquélla que invocó la subrutina.

Palabras clave asociadas: GOSUB.

RIGHT\$

Sintaxis: RIGHT\$ (<expresión literal> , <longitud deseada>)

Ejemplo:

```
10 MODE 0:a$="Ordenador CPC6128"
20 FOR n=1 TO 17:LOCATE 21-n,n
30 PRINT RIGHT$(a$,n)
40 NEXT
run
```

Función. Produce una cadena que consta del número de caracteres dado por (longitud deseada> (margen de 0 a 255) tomados de la derecha de la <expresión literal>. Si la longitud de la <expresión literal> es menor que la <longitud deseada>, la cadena producida es la <expresión literal> completa.

Palabras clave asociadas: LEFT\$, MID\$.

RND

Sintaxis: RND [(<expresión numérica>)]

Ejemplo:

```
10 RANDOMIZE
20 FOR x=1 TO -1 STEP -1
30 PRINT "parametro de rnd =" ;x
40 FOR n=1 TO 6
50 PRINT RND(X)
60 NEXT n,x
run
```

Función. Da el siguiente número aleatorio de la sucesión en el caso de que la <expresión

numérica> tenga valor positivo o no haya sido especificada.

Si la <expresión numérica> toma valor cero, RND vuelve a dar el anterior número aleatorio generado.

Si la <expresión numérica> toma un valor negativo, se inicia una nueva sucesión aleatoria y RND da su primer número.

Palabras clave asociadas: RANDOMIZE.

ROUND

Sintaxis: ROUND (<expresión numérica> [,<decimales>])

Ejemplo:

```
10 FOR n=4 TO -4 STEP-1
20 PRINT ROUND(1234.5678,n),
30 PRINT "con <decimales> =" ;n
40 NEXT
run
```

Función. Redondea la <expresión numérica> a un número de decimales o potencia de diez especificada por <decimales>. Si <decimales> es menor que cero, se redondea la <expresión numérica> para dar un número entero seguido de tantos ceros antes del punto decimal como indique el parámetro <decimales>.

Palabras clave asociadas: ABS, CINT, FIX, INT.

RUN

Sintaxis: RUN <expresión literal>

Ejemplo:

```
RUN "disc"
```

Orden. Carga un programa de BASIC o un programa binario que esté grabado en disco e inicia su ejecución. Borra el programa anteriormente residente en la memoria.

Los programas de BASIC protegidos pueden ser ejecutados por este procedimiento.

Palabras clave asociadas: LOAD.

RUN

Sintaxis: RUN [<número de línea>]

Ejemplo:

```
RUN 200
```

Orden. Inicia la ejecución del programa de BASIC actual a partir de la línea especificada por <número de línea>, o a partir de la primera línea del programa si se omite el parámetro.

Los programas protegidos no pueden ser ejecutados por este procedimiento.

Palabras clave asociadas: CONT, END, STOP.

SGN

Sintaxis: SGN (<expresión numérica>)

Ejemplo:

```
10 FOR n=200 TO -200 STEP-20
20 PRINT "SGN da";
30 PRINT SGN(n);"cuando el numero es";n
40 NEXT
run
```

Función. Determina el signo de la <expresión numérica>. Da el valor -1 si la <expresión numérica> es menor que cero; el valor 0 si es igual a 0; y el valor 1 si es mayor que cero.

Palabras clave asociadas: ABS.

SIN

Sintaxis: SIN (<expresión numérica>)

Ejemplo:

```
10 CLS:DEG:ORIGIN 0,200
20 FOR n=0 TO 720
30 y=SIN(n)
40 PLOT n*640/720,198*y:NEXT
50 GOTO 50
run
```

Función. Calcula el seno de la <expresión numérica>.

Obsérvese que se puede utilizar DEG y RAD para hacer que el ordenador entienda los arcos como dados en grados o en radianes, respectivamente.

Palabras clave asociadas: ATN, COS, RAD, TAN.

SOUND

Sintaxis: SOUND <situación de canales>, <periodo de tono>

[,<duración> [<volumen> [<envolvente de volumen>

[,<envolvente de tono>[,<ruido>]]]]]

Ejemplo:

```
10 FOR z=0 TO 4095
20 SOUND 1,z,1,12
30 NEXT
run
```

Orden. Programa la generación de un sonido. Los parámetros son los siguientes:

Parámetro 1: <situación de canales>

El parámetro <situación de canales> debe tener un valor comprendido entre 1 y 255.

El significado de cada uno de los bits es como sigue:

- Bit 0 (decimal 1) enviar sonido al canal A (bit menos significativo)

- Bit 1 (decimal 2) enviar sonido al canal B
- Bit 2 (decimal 4) enviar sonido al canal C
- Bit 3 (decimal 8) sincronización con el canal A
- Bit 4 (decimal 16) sincronización con el canal B
- Bit 5 (decimal 32) sincronización con el canal C
- Bit 6 (decimal 64) retención del canal de sonido
- Bit 7 (decimal 128) borrado del canal de sonido (bit más significativo)

Por ejemplo, el valor 68 del parámetro <situación de canales> especifica: Enviar el sonido al canal C (4) y retenerlo (64).

Parámetro 2: <periodo de tono>

Este parámetro define el tono del sonido; es decir, identifica la nota (DO, RE, MI, FA, etc.). Cada nota tiene un número de identificación, que es el <periodo de tono >. Véase el capítulo titulado 'Para su referencia...!.

Parámetro 3: <duración>

Este parámetro especifica la duración del sonido. Las unidades son de centésimas de segundo (0.01 s). Si no se especifica duración, el ordenador toma el valor implícito para este parámetro, que es 20 (un quinto de segundo).

Si el valor de <duración> es cero, el sonido dura hasta el final de la envolvente de volumen especificada.

Si el valor de <duración> es negativo, la envolvente de volumen especificada se repite ABS (<duración>) veces.

Parámetro 4: <volumen>

Este parámetro especifica el volumen inicial de la nota. El margen de valores va de 0 a 15. El <volumen> 0 es el mínimo; el 15 es el máximo. Si no se especifica <volumen>, el ordenador toma el implícito, que es 12.

Parámetro 5: <envolvente de volumen>

Se puede hacer que el volumen de la nota no sea constante, sino que varíe con el tiempo mientras la nota está sonando. Para especificar la forma de variación del volumen con el tiempo se utiliza la orden ENV. De hecho, se pueden crear hasta 15 envolventes de volumen diferentes, con números de referencia del 1 al 15. El parámetro <envolvente de volumen> invoca una envolvente de volumen para su utilización por SOUND.

Véase la descripción de la orden ENV.

Parámetro 6: <envolvente de tono>

Se puede hacer que el tono de una nota no sea constante, sino que varíe con el tiempo mientras la nota está sonando. Para especificar la forma de variación del tono con el tiempo se utiliza la orden ENT. De hecho, se pueden crear hasta 15 envolventes de tono diferente, con números de referencia del 1 al 15. El parámetro <envolvente de tono> invoca una envolvente de tono para su utilización por SOUND. Si en la definición de ENT se ha puesto un número negativo como número de envolvente, en la instrucción SOUND se debe poner el valor absoluto de ese número como <envolvente de tono>.

Véase la descripción de la orden ENT.

Parámetro 7: <ruido>

Se puede añadir ruido al sonido: la intensidad del ruido se controla con el parámetro <ruido>. El margen de valores va de 0 (ruido apagado) hasta 31.

En la parte 2 del capítulo 'Cuando usted guste ...' daremos más amplia información sobre los sonidos.

Palabras clave asociadas: ENT, ENV, ON SQ GOSUB, RELEASE, SQ.

SPACE\$

Sintaxis: SPACE\$ (<expresión entera>)

Ejemplo:

```
10 MODE 1
20 PRINT "Pongamos 5 espacios entre uno";
30 PRINT SPACE$(5);
40 PRINT "y otro"
run
```

Función. Crea una cadena literal consistente en el número de espacios especificado por <expresión entera> (margen de 0 a 255).

Palabras clave asociadas: SPC, STRING\$, TAB.

SPC

Véase PRINT SPC.

SPEED INK

Sintaxis: SPEED INK <periodo 1>, <periodo 2>

Ejemplo:

```
10 BORDER 7,18
20 FOR i=30 TO 1 STEP-1
30 SPEED INK i,i
40 FOR t=1 TO 700:NEXT t,i
run
```

Orden. Establece la velocidad de alternancia entre los dos colores de tinta especificados mediante INK o BORDER. El primer parámetro, periodo 1>, especifica el tiempo para el primer color, en unidades de 0.02 s (cincuentavos de segundo); <periodo 2> especifica el tiempo para el segundo color.

Téngase cuidado al elegir colores y velocidades para evitar efectos mesméricos ('hipnotizantes').

Palabras clave asociadas: BORDER, INK.

SPEED KEY

Sintaxis: SPEED KEY <demora de arranque>, <periodo de repetición>

Ejemplo:

```
10 CLS:FOR k=7 TO 1 STEP-2
20 PRINT "Escriba su nombre y pulse [RETURN]"
30 SPEED KEY k,k
```

```

40 LINE INPUT a$:NEXT
50 PRINT "Es un nombre muy interesante"
run

```

Orden. Establece las características de repetición de las teclas. El parámetro <demora de arranque> especifica el tiempo, en unidades de 0.02 s (cincuentavos de segundo) que ha de estar pulsada la tecla antes de que empiece a repetirse su acción. El parámetro <periodo de repetición> especifica el tiempo que debe transcurrir entre cada dos repeticiones sucesivas.

SPEED KEY solamente actúa sobre las teclas que son de repetición implícitamente o que han sido definidas como tales mediante KEY DEF.

Cuando se vaya a utilizar valores muy bajos del parámetro demora de arranque>, es aconsejable definir previamente una de las teclas numéricas para que devuelva el teclado a la situación implícita, SPEED KEY 30,2. La orden

```
KEY 0,"SPEED KEY 30,2"+CHR$(13)
```

restablece los valores implícitos de los parámetros de SPEED KEY en cuanto se pulsa la tecla del 0 del teclado numérico.

Palabras clave asociadas: KEY DEF.

SPEED WRITE

Sintaxis: SPEED WRITE <expresión entera>

Ejemplo:

```
SPEED WRITE 1
```

Orden. Establece la velocidad de transferencia de datos al magnetófono (si está conectado). Las velocidades son: 2000 bits por segundo, si <expresión entera> vale 1, y 1000 bits por segundo si <expresión entera> vale 0. Esta última es la velocidad implícita. Para leer una cinta no es necesario saber a qué velocidad ha sido grabada, pues el ordenador selecciona automáticamente la velocidad correcta de lectura.

La máxima fiabilidad en la transferencia de datos se obtiene con la velocidad mínima, que además es la implícita: SPEED WRITE 0.

Esta orden no ejerce ningún efecto sobre el sistema de grabación en disco.

Palabras clave asociadas: OPENOUT, SAVE.

SQ

Sintaxis: SQ (<canal>)

Ejemplo:

```

10 SOUND 65,100,100
20 PRINT SQ(1)
run
67

```

Función. Genera un número que informa del estado de la cola de sonido del <canal> especificado. El parámetro puede tener uno de los siguientes valores:

- 1 para el canal A

- 2 para el canal B
- 4 para el canal C

La función SQ produce un número que se interpreta, según sea el valor de sus bits, de la forma siguiente:

- Bits 0, 1 y 2: número de huecos libres en la cola
- Bits 3, 4 y 5: estado de sincronización de la primera nota de la cola
- Bit 6 la primera nota está retenida
- Bit 7 el canal está activo en la actualidad

donde el bit 0 es el menos significativo y el bit 7 el más significativo.

Como se puede apreciar, si el bit 6 está a 1, el bit 7 tiene que estar a cero, y viceversa. Análogamente, si alguno de los bits 3, 4 y 5 está a 1, los bits 6 y 7 tienen que estar a cero.

En la parte 2 del capítulo 'Cuando usted guste ...' daremos más información acerca de los sonidos.

Palabras clave asociadas: ON SQ GOSUB, SOUND.

SQR

Sintaxis: SQR (<expresión numérica>)

Ejemplo:

```
PRINT SQR (9)
3
```

Función. Da el valor de la raíz cuadrada de la <expresión numérica>.

Palabras clave asociadas: ninguna.

STEP

Véase FOR.

STOP

Sintaxis: STOP

Ejemplo:

```
10 FOR n=1 TO 30:PRINT n:NEXT
20 STOP
30 FOR n=31 TO 60:PRINT n:NEXT
run
cont
```

Orden. Detiene la ejecución del programa, pero deja BASIC en un estado tal, que el programa puede ser reanudado mediante CONT. STOP se utiliza para establecer puntos de parada en la depuración de programas.

Palabras clave asociadas: CONT, END.

STR\$

Sintaxis: STR\$ (<expresión numérica>)

Ejemplo:

```
10 a=&FF :REM 255 en hexadecimal
20 b=&X1111 :REM 15 en binario
30 c$="***"
40 PRINT c$+STR$(a+b)+c$
run
*** 270X**
```

Función. Convierte la <expresión numérica> en una cadena literal formada por los mismos dígitos decimales.

Palabras clave asociadas: BIN\$, DEC\$, HEX\$, VAL.

STRING\$

Sintaxis: STRING\$ (<longitud> , <especificador del carácter>)

Ejemplo:

```
PRINT STRING$(40, "*")
*****
```

Función. Da una cadena literal que consiste en el carácter especificado repetido tantas veces como indique <longitud> (margen de 0 a 255). Nótese que en el ejemplo anterior se podría haber puesto también:

```
PRINT STRING$(40, 42)
*****
```

donde el <especificador del carácter> , el número 42, es el código ASCII del asterisco.

Palabras clave asociadas: SPACE\$.

SWAP

Véase WINDOW SWAP.

SYMBOL

Sintaxis: SYMBOL <número del carácter> , <lista de:<fila>

Ejemplo:

```
10 MODE 1:SYMBOL AFTER 105
20 fila1=255:REM 11111111 en binario
30 fila2=129:REM 10000001 en binario
40 fila3=189:REM 10111101 en binario
50 fila4=153:REM 10011001 en binario
60 fila5=153:REM 10011001 en binario
70 fila6=189:REM 10111101 en binario
80 fila7=129:REM 10000001 en binario
90 fila8=255:REM 11111111 en binario
100 PRINT "La línea 110 redefine la letra i (105). Teclee varias veces esa
    letra y luego liste el programa."
110 SYMBOL 105,fila1, fila2, fila3, fila4, fila5, fila6, fila7, fila8
```

run

Orden. Redefine la forma de un carácter. El margen para todos los parámetros es de 0 a 255.

Con objeto de reservar espacio en la memoria del ordenador para la descripción del carácter, es necesario ejecutar previamente la orden:

```
SYMBOL AFTER x
```

donde x debe ser igual o menor que el número del carácter que se va a definir.

A continuación se emite la orden SYMBOL seguida, en primer lugar, por el número del carácter.

El carácter puede ser directamente accesible por el teclado; en cualquier caso se lo puede escribir mediante la orden

```
PRINT CHR$ (x)
```

Después de SYMBOL x viene una sucesión de hasta 8 parámetros, cada uno de los cuales describe una fila de la matriz del carácter. El margen de estos parámetros es de 0 a 255. La representación binaria de cada parámetro indica la situación de 'encendido' o 'apagado' de cada uno de los puntos de la fila correspondiente.

Por ejemplo, si el primero de los ocho parámetros vale 1, la fila superior del carácter tiene la representación binaria 00000001. Cada 1 indica que el correspondiente punto del carácter se ilumina con el color de PEN; cada 0 indica que el correspondiente punto no será visible, pues se lo escribe con el color de PAPER. Así pues, la primera fila de este carácter consiste en un punto en el extremo superior derecho. Los siguientes parámetros son 3,7,15,31,63,0,0. La representación binaria de los ocho parámetros es como sigue:

- parámetro (fila) 1: 00000001 en binario (1 en decimal)
- parámetro (fila) 2: 00000011 en binario (3 en decimal)
- parámetro (fila) 3: 00000111 en binario (7 en decimal)
- parámetro (fila) 4: 00001111 en binario (15 en decimal)
- parámetro (fila) 5: 00011111 en binario (31 en decimal)
- parámetro (fila) 6: 00111111 en binario (63 en decimal)
- parámetro (fila) 7: 00000000 en binario (0 en decimal)
- parámetro (fila) 8: 00000000 en binario (0 en decimal)

Observando el conjunto de las representaciones binarias podemos hacernos una idea del aspecto que tendrá el carácter. Asignemos estos parámetros al carácter número 255 mediante la orden:

```
SYMBOL 255,1,3,7,15,31,63,0,0
```

Al ser 0 los dos últimos, podemos omitirlos:

```
SYMBOL 255,1,3,7,15,31,63
```

Los parámetros se pueden introducir en forma binaria, con lo que se ahorra el trabajo de convertirlos previamente a la forma decimal. (Recuérdese que el prefijo para números binarios es &X.) Por ejemplo:

```
SYMBOL 255, &X00000001, &X00000011, &X00000111, &X00001111, &X00011111,  
&X00111111
```

Finalmente, para ver el resultado de la definición, ejecute la orden

```
PRINT CHR$(255)
```

Si se redefine uno de los caracteres accesibles por el teclado, el nuevo carácter aparecerá en la pantalla cada vez que se pulse la tecla correspondiente o se lo escriba con `PRINT CHR$( )`. Por otra parte, BASIC reconoce la forma del nuevo carácter cuando trata de leerlo en la pantalla, y no lo rechaza por incomprensible.

En la parte 2 del capítulo 'Cuando usted guste ...' daremos más información sobre la redefinición de caracteres.

Palabras clave asociadas: COPYCHR\$, HIMEM, MEMORY, SYMBOL AFTER.

SYMBOL AFTER

Sintaxis: SYMBOL AFTER <expresión entera>

Ejemplo:

```
10 CLS
20 SYMBOL AFTER 115
30 PRINT "La línea 40 redefine la letra s ";
40 SYMBOL 115,0,56,64,64,48,8,8,112
50 PRINT "a s"
60 PRINT "Cancele esta definicion de s"
70 PRINT "tecleando SYMBOL AFTER 240"
run
```

Orden. Establece el número de caracteres redefinibles por el usuario (en el margen de 0 a 256). El número implícito es 240, lo que deja 16 caracteres (del 240 al 255) definibles por el usuario. Si <expresión entera> es 32, todos los caracteres, del 32 al 255, son definibles. SYMBOL AFTER 256 impide la redefinición de caracteres.

Cada vez que se ejecuta una orden SYMBOL AFTER, quedan anuladas todas las redefiniciones y la forma de todos los caracteres vuelve a ser la implícita.

La orden SYMBOL AFTER no se puede ejecutar después de modificar el valor de HIMEM, lo cual puede ocurrir como resultado de una orden MEMORY, o bien al abrir un tampón para la grabación o lectura de un fichero con OPENOUT u OPENIN. En tales circunstancias se provoca el mensaje 'improper argument' ('argumento inadecuado', error número 5), a menos que la situación anterior fuera SYMBOL AFTER 256.

En la parte 2 del capítulo 'Cuando usted guste...' daremos más información sobre la redefinición de caracteres.

Palabras clave asociadas: HIMEM, MEMORY, SYMBOL.

TAB

Véase PRINT TAB.

TAG

Sintaxis: TAG [# <expresión de número de canal>]

Ejemplo:

```

10 INPUT "Escriba su nombre: ",a$:CLS
20 PRINT "Se mueve usted mucho, ";a$
30 TAG
40 x=LEN(a$)*17:y=50+RND*300:MOVE -x,y
50 FOR f=-x TO 640 STEP RND*7+3
60 MOVE f,y:PRINT " ";a$;:FRAME:NEXT
70 FOR b=640 TO -x STEP-RND*7+3
80 MOVE b,y:PRINT a$;" ";:FRAME:NEXT
90 GOTO 40
run

```

Orden. Hace que el texto enviado al canal especificado sea escrito en la pantalla gráfica, en la posición actual del cursor. De esta forma se puede mezclar texto con gráficos, o mover los caracteres pixel a pixel. Si se omite la <expresión de número de canal>, se supone el #0.

El punto superior izquierdo del carácter se conecta al cursor gráfico. Si se omite el signo de punto y coma al final de la instrucción PRINT, los códigos de retorno del carro y de avance de línea se visualizan como caracteres gráficos.

Si se ha aplicado TAG al canal #0, BASIC desactiva TAG al volver a modo directo.

Palabras clave asociadas: TAGOFF.

TAGOFF

Sintaxis: TAGOFF [# <expresión de número de canal>]

Ejemplo:

```

10 MODE 2:TAG :REM texto ligado al cursor grafico
20 periodo=1984:FOR x=1 TO 640 STEP 70
30 MOVE x,400:DRAWR 0,-350
40 periodo=periodo+1:PRINT periodo;:NEXT
50 TAGOFF :REM texto desconectado de cursor grafico
60 LOCATE 34,25:PRINT "Datos anuales"
70 GOTO 70
run

```

Orden. Cancela la orden TAG para el canal especificado (o para el #0, si no se especifica otro) y redirige el texto a la posición que tenía el cursor de texto antes de ejecutarse TAG.

Palabras clave asociadas: TAG.

TAN

Sintaxis: TAN (<expresión numérica>)

Ejemplo:

```

PRINT TAN(45)
1.61977519

```

Función. Calcula la tangente de la <expresión numérica>, que debe tener un valor comprendido entre -20000 y +20000.

Obsérvese que se puede utilizar DEG y RAD para hacer que el ordenador entienda los arcos como dados en grados o en radianes, respectivamente.

Palabras clave asociadas: ATN, COS, DEG, RAD, SIN.

TEST

Sintaxis: TEST (<coordenada x>, <coordenada y>)

Ejemplo:

```
10 CLS
20 PRINT "Estamos utilizando la pluma numero";
30 PRINT TEST(10,386)
40 PRINT "Cambie la pluma y el modo ";
50 PRINT "... y ejecute otra vez el programa."
Run
```

Función. Traslada el cursor gráfico al punto de coordenadas absolutas especificadas e informa del valor de la tinta presente en esa posición.

Palabras clave asociadas: MOVE, MOVER, TESTR, XPOS, YPOS.

TESTR

Sintaxis: TESTR (<incremento de x>, <incremento de y>)

Ejemplo:

```
10 MODE 0:FOR x=1 TO 15:LOCATE 1,x
20 PEN x:PRINT STRING$(10,143);:NEXT
30 MOVE 200,400:PEN 1
40 FOR n=1 TO 23:LOCATE 12,n
50 PRINT "pen";TESTR(0,-16):NEXT
run
```

Función. Lleva el cursor gráfico a un punto cuya posición, referida a la posición actual, viene dada por los parámetros <incremento de x> e <incremento de y>, e informa del valor de la tinta presente en ese punto.

Palabras clave asociadas: MOVE, MOVER, TEST, XPOS, YPOS.

THEN

Véase IF.

TIME

Sintaxis: TIME

Ejemplo:

```
10 CLS:REM reloj
20 INPUT "Hora: ",hora
30 INPUT "Minuto: ",minuto
40 INPUT "Segundo: ",segundo
50 CLS:referencia=INT(TIME/300)
60 WHILE hora<13
70 WHILE minuto<60
80 WHILE tiempo<60
90 tiempo=(INT(TIME/300)-referencia)+segundo
100 LOCATE 1,1
110 PRINT USING "## ";hora,minuto,tiempo
120 WEND
```

```

130 tiempo=0:segundo=0:minuto=minuto+1
140 GOTO 50
150 WEND
160 minuto=0:hora=hora+1
170 WEND
180 hora=1
190 GOTO 60

```

Función. Da el tiempo transcurrido desde el momento en que se encendió o reinició la máquina por última vez (excluyendo el dedicado a escribir o leer en el disco), en unidades de 1/300 s. Expresado en segundos, el tiempo es, pues, TIME/300.

Palabras clave asociadas: AFTER, EVERY, WEND, WHILE.

TO

Véase FOR.

TROFF

TRON

Sintaxis:

TROFF

TRON

Ejemplo:

```

10 TROFF:PRINT:PRINT "TROFF"
20 FOR n=1 TO 8
30 PRINT "Programa funcionando":NEXT
40 IF f=1 THEN END
50 TRON:PRINT:PRINT "TRON"
60 f=1:GOTO 20
run

```

Orden. Sigue la 'traza' del programa escribiendo los números de cada línea antes de ejecutarla. Los números de línea aparecen entre corchetes [].

TRON activa este modo; TROFF lo desactiva.

TRON es particularmente útil para estudiar la secuencia de números de línea anterior al momento en que se produce un error.

Palabras clave asociadas: ninguna.

UNT

Sintaxis: UNT (<expresión de dirección de memoria>)

Ejemplo:

```

PRINT UNT (&FF66)
-154

```

Orden. Da un entero, comprendido entre -32768 y +32767, que es el equivalente en

'complemento a dos' al valor de <expresión de dirección de memoria>.

Palabras clave asociadas: CINT, FIX, INT, ROUND.

UPPER\$

Sintaxis: UPPER\$ (<expresión literal>)

Ejemplo:

```
10 CLS:a$="mire como crecen!"
20 PRINT UPPER$(a$)
run
```

Función. Da una cadena literal que es copia de la <expresión literal> especificada, pero convirtiendo las letras minúsculas en mayúsculas y dejando inalterados los restantes caracteres. Esta función es útil en el proceso de datos que puedan contener minúsculas mezcladas con mayúsculas.

Palabras clave asociadas: LOWER\$.

USING

Véase PRINT USING.

VAL

Sintaxis: VAL (<expresión literal>)

Ejemplo:

```
10 CLS:PRINT "Ya he aprendido la tabla de multiplicar"
20 PRINT:PRINT "Pulse una tecla del 1 al 9:"
30 a$=INKEY$:IF a$="" THEN 30
40 n=VAL(a$):IF n<1 OR n>9 THEN 30
50 FOR x=1 TO 10
60 PRINT n;"x";x;"=":n*x
70 NEXT:GOTO 20
run
```

Función. Da el valor numérico (incluyendo el signo y el punto decimal) de los primeros caracteres de la <expresión literal> que sean dígitos.

Si el primer carácter no es un dígito, el valor generado es 0. Si el primer carácter es el signo menos o el punto decimal y va seguido de caracteres no numéricos, se provoca el mensaje "Type mismatch" ('incongruencia de tipos', error número 13).

Palabras clave asociadas: STR\$.

VPOS

Sintaxis: VPOS (# <expresión de número de canal>)

Ejemplo:

```
10 MODE 1:BORDER 0:LOCATE 8,2
20 PRINT "Utilice las teclas subir/bajar cursor"
30 WINDOW 39,39,1,25:CORSOR 1,1
40 LOCATE 1,13
```

```

50 IF INKEY(0)<>-1 THEN PRINT CHR$(11);
60 IF INKEY(2)<>-1 THEN PRINT CHR$(10);
70 LOCATE #1,1,23
80 PRINT #1,"Posicion vertical del"
90 PRINT #1,"cursor de texto =";
100 PRINT #1,VPOS(#0):GOTO 50
run

```

Función. Da la posición vertical del cursor de texto, referida al extremo superior de la ventana. Es necesario especificar la <expresión de número de canal>, pues el ordenador no da por supuesto el #0.

Palabras clave asociadas: POS, WINDOW.

WAIT

Sintaxis: WAIT <número de puerta>, <máscara>[, <inversión>]

Ejemplo:

```
WAIT &FF34,20,25
```

Orden. Espera hasta que por la puerta de entrada/salida especificada por <número de puerta> se reciba un determinado valor. BASIC lee incesantemente la puerta. Con el valor recibido y el parámetro <inversión> se realiza la operación lógica XOR; con este resultado y el parámetro <máscara> se realiza la operación lógica AND; el proceso se repite hasta que el resultado final sea distinto de cero.

BASIC espera indefinidamente hasta que se cumple la condición requerida. Esta orden no debe ser utilizada por quien no sepa cuáles son sus efectos.

Palabras clave asociadas: INP, OUT.

WEND

Sintaxis: WEND

Ejemplo:

```
WEND
```

Orden. Señala el final del grupo de instrucciones cuya ejecución se repite en un bucle del tipo WHILE. BASIC determina automáticamente a qué WHILE corresponde cada WEND.

Palabras clave asociadas: TIME, WHILE.

WHILE

Sintaxis: WHILE <expresión lógica>

Ejemplo:

```

10 CLS:PRINT "Temporizador de 10 segundos":t=TIME
20 WHILE TIME< t+3000
30 SOUND 1,0,100,15
40 WEND: SOUND 129,40,30,15
run

```

Orden. Repite la ejecución de un grupo de instrucciones mientras se cumpla cierta condición. La

orden WHILE señala el principio del bucle y especifica la condición lógica mediante la <expresión lógica>.

Palabras clave asociadas: TIME, WEND.

WIDTH

Sintaxis: WIDTH <expresión entera>

Ejemplo:

```
WIDTH 40
```

Orden. Informa a BASIC de cuántos caracteres por línea se deben enviar a la impresora. BASIC envía a la impresora los códigos de retorno del carro y avance de línea en el momento adecuado.

El valor implícito es 132.

La orden WIDTH 255 suprime los códigos de retorno del carro y avance de línea, dejando que el número de caracteres por línea sea totalmente controlado por la impresora. En todo caso, la orden PRINT envía a la impresora esos dos códigos siempre que no se incluya el signo de punto y coma al final.

Palabras clave asociadas: POS.

WINDOW

Sintaxis: WINDOW [<#> <expresión de número de canal>,<izquierda>, <derecha>,<arriba>, <abajo>

Ejemplo:

```
10 MODE 0:BORDER 0:REM carta de ajuste
20 INK 0,0:INK 1,25: INK 2,23:INK 3,21
30 INK 4,17:INK 5,6:INK 6,2:INK 7,26
40 PAPER 0:CLS
50 PAPER 1:WINDOW 2,4,1,18:CLS
60 PAPER 2:WINDOW 5,7,1,18:CLS
70 PAPER 3:WINDOW 8,10,1,18:CLS
80 PAPER 4:WINDOW 11,13,1,18:CLS
90 PAPER 5:WINDOW 14,16,1,18:CLS
100 PAPER 6:WINDOW 17,19,1,18:CLS
110 PAPER 7:WINDOW 2,19,19,25:CLS
120 GOTO 120
run
```

Orden. Especifica las dimensiones y situación de una ventana de texto. Los valores de los parámetros <izquierda>, <derecha>, <arriba> y <abajo> indican los números extremos de columna y de fila, y deben ser consistentes con el modo de pantalla actual.

Si no se especifica la <expresión de número de canal>, BASIC supone el #0.

En la parte 2 del capítulo titulado 'Cuando usted guste...' daremos más información sobre ventanas de texto.

Palabras clave asociadas: WINDOW SWAP.

WINDOW SWAP

Sintaxis: WINDOW SWAP <expresión de número de canal>,
<expresión de número de canal>

Ejemplo:

```
10 MODE 1:INK 1,24:INK 2,9:INK 3,6
20 WINDOW 21,40,13,25:PAPER 3
30 WINDOW #1,1,20,1,12:PAPER #1,2
40 CLS: PRINT " Ventana numero 0"
50 CLS #1:PRINT #1," Ventana numero 1"
60 LOCATE 1,6
70 PRINT " Ventana roja (0)";SPC(2)
80 LOCATE #1,1,6
90 PRINT #1," Ventana verde (1)"
100 FOR t=1 TO 1000:NEXT
110 WINDOW SWAP 0,1:GOTO 60
run
```

Orden. Intercambia las características de las dos ventanas de texto especificadas por las <expresiones de número de canal>.

Se deben especificar los dos números de canal, pero en este caso no deben ir precedidos del signo #.

En la parte 2 del capítulo titulado 'Cuando usted guste...' daremos más información sobre ventanas de texto.

Palabras clave asociadas: WINDOW.

WRITE

Sintaxis: WRITE [#<expresión de número de canal> ,][<lista de datos>]

Ejemplo:

```
10 REM escribir variables en disco
20 INPUT "Escriba un numero: ",a
30 INPUT "Escriba una palabra: ",a$
40 OPENOUT "datos"
50 WRITE #9,a,a$
60 CLOSEOUT:PRINT "Datos grabados en disco"
run
```

Orden. Envía los valores de los elementos de la <lista de datos> al canal especificado por la <expresión de número de canal>. Los elementos irán separados por comas; las cadenas literales irán entre comillas.

En el ejemplo anterior, los valores captados por el teclado son enviados al canal #9 (disco).

Para leer esos valores después de grabarlos en disco, se necesita un programa tal como el siguiente:

```
10 REM leer variables del disco
20 OPENIN "datos":INPUT #9,a,a$
30 CLOSEIN:PRINT "Los dos valores son:"
40 PRINT:PRINT a,a$
run
```

Palabras clave asociadas: INPUT, LINE INPUT.

XOR

Sintaxis: <argumento> XOR <argumento>

Ejemplo:

```
IF "juan"<"pepe" XOR "perro">"gato" THEN PRINT "verdadero" ELSE PRINT "falso"
falso
IF "pepe"<"juan" XOR "gato">"perro" THEN PRINT "verdadero" ELSE PRINT "falso"
falso
IF "juan"<"pepe" XOR "gato">"perro" THEN PRINT "verdadero" ELSE PRINT "falso"
verdadero
...
PRINT 1 XOR 1
0
PRINT 0 XOR 0
0
PRINT 1 XOR 0
1
```

Operador. Realiza la operación lógica 'o exclusivo' bit a bit entre enteros. El resultado es 1 a menos que ambos bits tengan el mismo valor.

Para más amplia información sobre operaciones lógicas, consulte la parte 2 del capítulo titulado 'Cuando usted guste ...'.

Palabras clave asociadas: AND, OR, NOT.

XPOS

Sintaxis: XPOS

Ejemplo:

```
10 MODE 1:DRAW 320,200
20 PRINT "Posicion horizontal del cursor grafico =";
30 PRINT XPOS
run
```

Función. Da el valor de la posición horizontal <coordenada x> del cursor gráfico.

Palabras clave asociadas: MOVE, MOVER, ORIGIN, YPOS.

YPOS

Sintaxis: YPOS

Ejemplo:

```
10 MODE 1:DRAW 320,200
20 PRINT "Posicion vertical del cursor grafico =";
30 PRINT YPOS
run
```

Función. Da el valor de la posición vertical <coordenada y> del cursor gráfico.

Palabras clave asociadas: MOVE, MOVER, ORIGIN, XPOS.

ZONE

Sintaxis: ZONE <expresión entera>

Ejemplo:

```
10 CLS:FOR z=2 TO 20
20 ZONE z
30 PRINT "X","X ZONE =";z:NEXT
run
```

Orden. Cambia la anchura de la zona de escritura (especificada en las instrucciones PRINT cuando se utiliza una coma para separar elementos). La anchura implícita es 13 columnas, pero se la puede cambiar al valor dado por la <expresión entera>, cuyo margen es de 1 a 255.

Palabras clave asociadas: PRINT.

Nota del revisor (2008): Basic 1.1 frente a Basic 1.0

La versión de BASIC incluida en los Amstrad CPC 6128 y CPC664 (Locomotive Basic 1.1) tiene algunas ampliaciones frente a la versión de los CPC464 (Locomotive Basic 1.1), como:

- La función COPYCHR\$ no existía en los 464.
- La orden CURSOR no existía en los 464.
- La orden FILL no existía en los 464.
- La orden FRAME no existía en los 464, aunque se podía imitar con CALL &BD19.
- En los 464, las órdenes externas no permitían usar cadenas de texto directamente, sino que se debía utilizar una variable intermedia. Por ejemplo, en un 464 con unidad de disco externa, no se podía borrar un fichero con |ERA, "EJEMPLO" sino que había que hacerlo en dos pasos: a\$="EJEMPLO": |ERA, @a\$